

Master's Thesis

by

Yu Zhang

**Explain Reality: Grounded Procedural Tutoring
with Foundation Models
and Visual Fact Extraction in Immersive Simulation**

1st Supervisor: **Prof. Dr. Christian Bartelt**

2nd Supervisor: **Prof. Dr. Rüdiger Ehlers**

9th June 2026

Institute for Software and Systems Engineering
Clausthal University of Technology

Declaration of Authorship

I have read and understood the guidelines of the Technical University of Clausthal and those published in the ISSE bulletin, including those regarding the use of literature and other sources. I confirm that I have prepared this thesis independently by myself. Any information taken from other sources and being reproduced in this thesis is clearly referenced.

In terms of the general examination regulations, this work has not yet been submitted to any other examination division.

I hereby agree that my master's thesis may be exhibited in the institute's library and kept for inspection.

Clausthal-Zellerfeld, 9th June 2026

Location, Date

Yu Zhang

Abstract

Many safety- and skill-critical domains, including aviation, industrial maintenance, medicine, and emergency operations, require learners to practise ordered procedures whose next correct action depends on the current system state. As such training moves into high-fidelity and immersive simulation, tutoring systems must provide contextual help without relying on language-model fluency alone. This thesis studies that problem through the F/A-18C cold-start procedure in DCS World and designs an evidence-grounded tutoring runtime that integrates procedure-pack-driven state tracking, retrieval-augmented help generation, and a fine-tuned vision-language model (VLM) for structured visual fact extraction from cockpit displays. A central finding is that visual fact ontology design governs model reliability: evolving from an 8-fact to a 13-fact ontology by decomposing compound targets into locally verifiable visual atoms reduced critical false positives by 64–89% on independent holdout sets. A Qwen3.5-9B LoRA adapter trained on 928 bilingual rows achieves 0.99 fact accuracy on two holdout sets with verified zero training overlap; the identical data recipe transfers to Gemma-4-31B with consistent gains, suggesting the approach is not tied to a single backbone. A small formative VR pilot study with nine participants provides descriptive evidence: all four tutor-condition participants completed the full 33-step procedure, whereas none of the five without-tutor participants did; omission errors were largely replaced by assistance-coupled errors in the tutor condition. Controlled evaluation with larger samples remains future work.

Zusammenfassung

Prozedurales Lernen in hochrealistischer Simulation erfordert das Ausführen geordneter, zustandsabhängiger Handlungsabfolgen bei gleichzeitiger Überprüfung von Systemzuständen. Herkömmliche Trainingshilfen unterbrechen jedoch entweder die Immersion oder haben keinen Zugriff auf den Simulatorzustand. Diese Arbeit begegnet dieser Lücke durch eine telemetriegestützte Tutoring-Laufzeitumgebung, die paketbasierte Zustandsverfolgung, Retrieval-Augmented-LLM-Hilfegenerierung und ein feinabgestimmtes Vision-Language-Model (VLM) zur strukturierten Extraktion visueller Fakten aus Cockpit-Anzeigen kombiniert. Ein zentrales Ergebnis ist, dass das Design der visuellen Fakten-Ontologie die Modellzuverlässigkeit bestimmt: Die Weiterentwicklung von einer 8-Fact- zu einer 13-Fact-Ontologie durch Zerlegung zusammengesetzter Ziele in lokal überprüfbare visuelle Atome reduzierte kritische False-Positives um 64–89% auf unabhängigen Holdout-Sets. Ein Qwen3.5-9B LoRA-Adapter, trainiert mit 928 bilingualen Datenzeilen, erreicht eine Fact-Genauigkeit von 0,99 auf zwei Holdout-Sets mit verifizierter Null-Überlappung zum Training; dieselbe Datenrezeptur überträgt sich mit konsistenten Verbesserungen auf Gemma-4-31B, was darauf hindeutet, dass der Ansatz nicht an ein einzelnes Backbone gebunden ist. Eine kleine formative VR-Pilotstudie mit neun Teilnehmern liefert deskriptive Evidenz: Alle vier Teilnehmer mit Tutor absolvierten die vollständige 33-Schritt-Prozedur, während keiner der fünf Teilnehmer ohne Tutor dies schaffte; Auslassungsfehler wurden in der Tutor-Bedingung weitgehend durch assis-
tenzgekoppelte Fehler ersetzt. Eine kontrollierte Evaluation mit größeren Stichproben bleibt zukünftige Arbeit.

Acknowledgements

I would like to thank my supervisor for guidance and feedback throughout this thesis. I also thank the pilot study participants for their time, patience, and careful engagement during the VR sessions.

Declaration on the Use of AI Tools

AI-assisted tools, including ChatGPT/Codex and Claude Code, were used during the preparation of this thesis for language editing, restructuring suggestions, LaTeX consistency checks, citation-gap auditing, and inspection of repository artefacts under the author's direction. These tools were not used to fabricate experimental data, participant results, or bibliographic metadata. All claims, numerical results, citations, interpretations, and final wording remain the responsibility of the author.

Contents

1	Introduction	1
1.1	Motivation and Problem Context	1
1.2	Research Gap and Problem Statement	2
1.3	Research Questions	2
1.4	Approach and Scope	3
1.5	Contributions	4
1.6	Thesis Structure	6
2	Background and Related Work	7
2.1	The F/A-18C Cold-Start Task	7
2.1.1	Procedure Structure: S01–S33 and Phases P1–P6	7
2.1.2	Cockpit Displays and the Composite Panel	8
2.1.3	DCS-BIOS Telemetry	9
2.2	Visual Fact Ontology	9
2.2.1	From 8-Fact v1 to 13-Fact v2	11
2.2.2	Sticky Facts and Step Bindings	12
2.3	Intelligent Tutoring Systems and Procedural Training	13
2.4	Simulator-Based Training and Instrumentation	14
2.5	Rule-Based and State-Aware Tutoring	14
2.6	Foundation Models and VLMs for Situated Assistance	15
2.7	Explainable and Grounded AI Assistance in AR/VR	16
2.8	Positioning of This Work	16
3	System Design	18
3.1	Design Goals and Requirements	18
3.2	Architecture: Ports and Adapters	19
3.3	The Evidence-Grounded Help Cycle	20
3.3.1	Help Cycle Stages	20

3.3.2	Schema-Level Evidence Grounding	23
3.4	Telemetry Processing	24
3.5	Procedure Engine and Gating Rules	24
3.5.1	Procedure State Machine	24
3.5.2	Gating Rule Engine	24
3.5.3	Deterministic Fallback	25
3.5.4	Step Classification by Evidence Source	25
3.6	Harness Engineering and Evidence Validation	25
3.6.1	Harness Architecture	25
3.6.2	Fixture-Based Regression Testing	26
3.7	Simulator Adapter Layer	27
3.8	Event Logging and Replay	28
3.8.1	Current Limitations	28
3.9	VLM Adaptation for Visual State Understanding	28
3.9.1	Vision Fact Ontology (13-Fact v2)	29
3.9.2	VLM Adaptation Pipeline	29
3.9.3	Runtime Integration	29
4	Methodology	30
4.1	Procedure Runtime and Telemetry Grounding	30
4.1.1	Variable Resolution and Source-Missing Tracking	30
4.1.2	Gating Rule Evaluation	30
4.1.3	Delta Sanitisation and Aggregation	31
4.1.4	Deterministic Fallback	31
4.2	Knowledge Grounding and Source Policy	31
4.3	Visual Fact Data Pipeline	32
4.4	Dataset Curation	34
4.4.1	Dataset Overview	34
4.4.2	Ontology Evolution: 8-Fact v1 to 13-Fact v2	34
4.4.3	Training Recipe: Run-003 + Run-005 $\times$ 2	36
4.4.4	Holdout Independence	36
4.5	LoRA Fine-Tuning Configuration	37
4.5.1	Backbone Models and Comparison Strategy	37
4.5.2	Training Stack and Hyperparameters	37
4.5.3	Bilingual SFT and Composition Rebalance	38

4.6	Benchmark Protocol	38
4.6.1	Metrics	38
4.6.2	Base vs. LoRA Comparison	39
4.6.3	Benchmark Categories	39
4.6.4	Benchmark Artifacts	39
4.7	Human-Subject Pilot Methodology	40
4.7.1	Participants and Conditions	40
4.7.2	Task and Apparatus	40
4.7.3	Data Sources and Export Pipeline	41
4.7.4	Error Taxonomy and Coding Policy	41
4.7.5	Data-Quality Notes	41
4.7.6	Analysis Approach	42
4.8	Summary	42
5	Implementation	44
5.1	Technology Stack	44
5.2	Evidence Grounding Enforcement	44
5.3	Deterministic Fallback	45
5.4	VLM Visual Fact Extraction Runtime	46
5.5	Replay and Logging Infrastructure	46
5.6	Live Runtime Orchestration	47
6	Evaluation	48
6.1	RQ2: VLM Technical Evaluation	48
6.1.1	Dataset and Training Summary	48
6.1.2	Qwen3.5 13-Fact V2 Results	50
6.1.3	Gemma-4 and Qwen3.6 Backbone Comparison	50
6.1.4	Error Analysis	51
6.2	RQ1: System-Level Technical Readiness	52
6.2.1	Replay Evaluation	52
6.2.2	Harness Fixture Regression	52
6.2.3	Experiment Export Infrastructure	52
6.3	RQ3: Formative VR Pilot Study	53
6.3.1	Participant and Trial Overview	53
6.3.2	Procedural Errors	54
6.3.3	Tutor Interaction	54

6.3.4	Data-Quality Notes	55
6.3.5	Summary of Pilot Findings	56
7	Discussion	57
7.1	Evolution from Proposal to Implemented System	57
7.2	Interpretation of Technical Results	58
7.2.1	What the Benchmark Results Demonstrate	58
7.2.2	What the Benchmark Results Do NOT Demonstrate	59
7.2.3	The <code>ins_go</code> to <code>ins_ok_text</code> Decomposition as a Design Lesson	60
7.3	Interpretation of Pilot Findings	60
7.4	Ontology Design Lessons	62
7.4.1	Abstraction-Level Discipline	62
7.4.2	Decomposition of Compound Targets	63
7.4.3	Residual Failure Modes and Their Implications	64
7.4.4	Implications for Future Ontology Design	64
7.5	Limitations	65
7.5.1	Limitations of the Experimental Setup	65
7.5.2	Limitations of the System	66
7.5.3	Limitations of the Pilot Study	66
7.5.4	Limitations of the VLM Adaptation Results	67
7.5.5	Scope Summary	67
7.6	Future Work	68
7.6.1	Controlled Human-Subject Evaluation	68
7.6.2	Extending the Visual Fact Ontology to Other Procedures	68
7.6.3	Multi-Frame Temporal Reasoning	69
7.6.4	Online Adaptation from In-Situ Corrections	69
7.6.5	System-Level End-to-End Evaluation	69
7.6.6	Summary of Future Directions	70
8	Conclusion	71
8.1	Summary of Completed Work	71
8.2	Future Work	73
	References	75
9	Appendix	78
9.1	Full LoRA Hyperparameter Configuration	78

9.2	8-Fact V1 Baseline Benchmark	79
9.3	Dataset Fact Distributions	79
9.4	CLI Command Reference	80
9.5	Model Provider Configuration	81
9.6	Replay Evaluation Suite	81
9.7	Supplementary Architecture and Export Diagrams	81
9.8	VR Pilot Study — Supplementary Data	81

List of Figures

2.1	Evidence sources for the F/A-18C cold-start task. The 33-step procedure combines telemetry-visible steps, vision-priority display checks, and a small number of manual or out-of-layout confirmations; downstream tutoring decisions are grounded in the corresponding evidence source rather than in language-model output alone.	10
3.1	Evidence-grounded tutoring architecture. Simulator telemetry, composite-panel vision, procedure-pack configuration, retrieved knowledge, and model outputs are integrated through an evidence packet and checked by the harness before any overlay or tutor text reaches the cockpit.	21
3.2	Live evidence-grounded help cycle. Each help request is converted into a telemetry and vision observation, interpreted against procedure gates, grounded with retrieved knowledge, passed through the text LLM, validated by the harness, and then dispatched as overlay/text output with an event-log trace.	22
4.1	VLM adaptation pipeline. DCS composite screenshots are pre-labelled, human-reviewed, converted into bilingual supervised fine-tuning rows, adapted with LoRA, evaluated on independent holdout sets, and used to drive ontology revision.	32
4.2	Ontology evolution from 8-fact v1 to 13-fact v2. Compound targets such as <code>ins_go</code> and broad FCS-MC completion facts were decomposed into locally verifiable visual atoms; the procedure engine then combines those observations with telemetry, phase, and timing evidence.	35
6.1	Evaluation and data flow across research questions. VLM benchmark artifacts support RQ2, replay and harness logs support RQ1 technical readiness, and VR pilot exports support the descriptive RQ3 findings. . .	49

9.1	Harness and evidence validation. Candidate tutor actions are checked against schema constraints, UI allowlists, evidence consistency, and procedure state before an overlay action plan is accepted; unsafe or under-grounded actions are repaired, rejected, or downgraded to text-only guidance.	82
9.2	Deployment topology used by the prototype. The simulator, Quest 3 overlay path, local runtime, replay utilities, and remote model providers are separated so that live trials and offline analysis can use the same evidence and logging contracts.	83
9.3	Detailed experiment export pipeline. Runtime event logs and trial metadata are transformed into participant-level summaries, step coding, help-cycle records, quality gates, and study-level analysis tables.	84

List of Tables

4.1	Dataset inventory. All counts from <code>stats.json</code> files.	34
6.1	Summary of annotated visual fact datasets.	49
6.2	13-fact v2 benchmark: Qwen3.5-9B Base vs. LoRA on two holdout sets. .	50
6.3	13-fact v2 results across three backbones (LoRA models only).	51
6.4	Participant and trial overview. Observed duration is the wall-clock interval from first telemetry event to last; it is not a completion time measure for incomplete trials.	53
6.5	Manual procedural error counts per participant (from manual step coding). OM = omission, CO = commission, OR = order error, PA = prompting/ assistance error, SV = state violation.	54
6.6	Tutor interaction summary (with-tutor condition only).	55
7.1	Summary of future work directions.	70
9.1	Complete LoRA fine-tuning hyperparameters across three backbones. . .	78
9.2	8-fact v1 benchmark: Qwen3.5-9B Base vs. LoRA-v1 on Run-002 holdout (50 samples, EN).	79
9.3	Per-fact seen counts across training and holdout datasets (13-fact v2 ontology).	80
9.4	Replay evaluation cases.	81
9.5	Pilot participant experience profiles (self-reported).	82
9.6	Steps not completed in the without-tutor condition (manual coding). . .	83
9.7	Auto-coded vs. manual error counts per participant.	85

1 Introduction

1.1 Motivation and Problem Context

Many safety- and skill-critical domains require people to learn ordered procedures whose next correct action depends on the current state of a system. Examples include aircraft cockpit procedures, industrial maintenance, emergency operations, and medical or technical equipment workflows. In such settings, procedural knowledge is not only a list of actions. It also includes preconditions, intermediate checks, state-dependent branching, and recovery from partially completed or incorrectly completed steps.

High-fidelity and immersive simulation make these procedures available for practice without exposing learners to the risks and costs of the real environment. At the same time, fidelity creates a tutoring problem. A learner who is already inside a simulated cockpit, machine, or procedure room may need help that is specific to the current state rather than a general explanation of the task. If the learner has to leave the simulation to consult a manual, video, or checklist, the training flow is interrupted. If the learner instead asks a language model for help, the answer may be fluent but disconnected from the state that actually exists in the simulation.

This thesis studies that problem through the F/A-18C cold-start procedure in DCS World. The flight-simulation setting is used as a concrete and demanding case study, not as a claim that the research problem exists only in aviation. The case is useful because it makes the evidence problem visible: procedure progress depends on switch states, engine parameters, warning lights, cockpit display pages, and the order in which earlier actions were completed. Some of these states are directly observable through telemetry; others require visual inspection of cockpit displays; and a small number require manual confirmation or lie outside the display layout used by the vision pipeline.

The current procedure pack contains 33 ordered steps (S01–S33) grouped into six operational phases, from initial electrical power through engine start, avionics configuration, flight-control checks, and final pre-taxi configuration (Section 2.1). The benchmark and VLM evaluation reported in this thesis focus on the original 25-step core procedure;

the later extension to S33 adds further pre-taxi checks without changing the central evidence problem. This makes the cold-start a compact but representative case for state-dependent procedural tutoring in high-fidelity simulation.

1.2 Research Gap and Problem Statement

Existing training aids and recent foundation-model systems each address part of the problem but leave an important gap. Manuals, videos, and checklists provide authoritative procedural knowledge, yet they do not observe the learner’s current simulator state. Conventional simulator telemetry can expose many states, yet it does not by itself explain what the learner should do next or interpret display content that is not exported as data. Foundation models can generate flexible explanations, but their output is not reliable enough for procedural guidance unless it is constrained by explicit evidence.

The central premise of this thesis is that procedural assistance becomes more reliable and more auditable when every actionable tutoring output is tied to an explicit evidence source. In the system developed here, such evidence may be a telemetry variable, a procedure-gate evaluation, a retrieved manual or checklist snippet, or a VLM-derived visual fact. The language model is therefore not asked to infer procedure state from parametric knowledge alone. It operates inside a runtime that supplies state evidence, checks structured outputs, validates overlay targets, and records the resulting help cycle for replay and analysis.

This framing changes the role of the foundation model. The model is not the tutor by itself; it is one component inside an evidence-grounded procedural tutoring runtime. The research question is therefore architectural and methodological as much as it is generative: how should simulator telemetry, procedure rules, retrieval, and visual fact extraction be connected so that generated help remains state-aware, inspectable, and bounded by available evidence?

1.3 Research Questions

The thesis is guided by one main research question and three subordinate questions:

Main RQ. How can an evidence-grounded multimodal tutoring system support state-dependent procedural training in high-fidelity immersive simulation?

RQ1 — System architecture and evidence integration. How can simulator telemetry, retrieved procedural knowledge, and visual fact observations be integrated so that tutoring outputs are state-aware, auditable, and safe for procedural guidance?

RQ2 — VLM adaptation and ontology design. To what extent can a small-data LoRA-adapted VLM reliably extract structured cockpit visual facts, and how does visual fact ontology design affect critical false positives?

RQ3 — Formative VR pilot evidence. In a VR-based F/A-18C cold-start training task, what formative evidence does a small pilot study provide about task completion, procedural errors, and tutor interaction in a with-tutor versus without-tutor comparison?

RQ1 and RQ2 are addressed through the implemented runtime, replay and harness infrastructure, and technical benchmark evaluation. RQ3 is addressed through a completed formative VR pilot study ($n = 9$). The pilot is treated as descriptive evidence about feasibility, instrumentation, completion, and error patterns; it is not used to claim controlled learning gains, workload reduction, retention, or transfer.

1.4 Approach and Scope

The thesis addresses the research questions by building and evaluating an evidence-grounded tutoring runtime for the F/A-18C cold-start case. The approach combines four evidence channels. First, DCS-BIOS telemetry provides the most authoritative signal for many cockpit controls, lights, and numerical engine or system values. Raw UDP frames are normalised, sanitised, and aggregated into stable runtime variables so that procedure gates can distinguish between a false value and a missing source. Second, where telemetry is insufficient, a visual fact layer converts cockpit display captures into structured facts with states `seen`, `not_seen`, or `uncertain`. Third, retrieved procedural sources provide textual support for explanations. Fourth, procedure-pack gates define which evidence is required before a step can be considered available or complete.

The help cycle is deliberately constrained. The runtime assembles an evidence packet containing the current procedure candidates, telemetry state, visual observations, and retrieved sources before model generation occurs. The LLM/VLM output is then validated against runtime-generated JSON Schema constraints and mapped through allowlisted overlay targets. If the model path is unavailable or invalid, the system falls back to deterministic, text-only guidance. This design makes the help cycle auditable: a later

replay can inspect which evidence was available, which step was inferred, what the model returned, and how the response was validated before display.

The scope is intentionally bounded. The thesis validates the architecture and visual fact pipeline technically, and it reports a completed formative VR pilot study. It does not claim that the system has established controlled learning gains, workload reduction, retention, transfer, or generality across all procedural domains. The F/A-18C case is used to make the evidence-grounding problem concrete and measurable. Extending the approach to other aircraft, industrial equipment, medical training tasks, or emergency procedures would require new procedure packs, evidence mappings, and domain-specific validation.

Within that boundary, the thesis scope is threefold:

1. The design and implementation of an evidence-grounded tutoring runtime that combines DCS-BIOS telemetry, procedure-pack gates, retrieved procedural knowledge, VLM-derived visual facts, structured response validation, overlay display, logging, and replay (Chapters 3, 4, and 5);
2. The technical validation of the visual fact adaptation pipeline through independent holdout benchmarks and cross-backbone comparisons (Chapter 6, Part A);
3. A completed formative VR pilot study ($n = 9$) that provides descriptive evidence about task completion, error patterns, tutor interaction, and instrumentation readiness under with-tutor and without-tutor conditions (Chapter 6, Part B).

Procedure definitions, gates, telemetry mappings, visual fact bindings, and error taxonomies are kept in declarative procedure-pack configuration rather than embedded in prompts. This pack-based design keeps the architecture aligned with Clean / Hexagonal principles: domain rules remain separate from simulator adapters, vision adapters, model adapters, and export tooling.

1.5 Contributions

This thesis makes three contributions.

1. **An evidence-grounded procedural tutoring architecture.** The thesis presents a ports-and-adapters tutoring runtime in which each actionable help output must be traceable to telemetry, a procedure-gate evaluation, a retrieved source, or a

visual fact. The contribution is not merely that an LLM is connected to a simulator. It is that generated tutoring is placed inside a constrained help cycle with declarative procedure packs, schema-validated responses, allowlisted overlay targets, deterministic fallbacks, and replayable logs. This contribution addresses RQ1 and is described in Chapters 3 and 5, with validation evidence in Chapter 6, Section 6.2.

2. **A visual fact ontology and small-data VLM adaptation pipeline.** The thesis develops a 13-fact visual ontology for F/A-18C cockpit display state, evolved from an 8-fact baseline that exposed the weakness of asking a VLM to recognise compound procedural states directly. The revised ontology decomposes such targets into locally observable visual atoms and leaves procedural interpretation to downstream rules. The associated data pipeline spans DCS screenshot capture, AI pre-labelling, human review, bilingual SFT row generation, LoRA fine-tuning, and independent holdout benchmarking. On the Qwen backbone, a LoRA adapter trained on 928 bilingual rows reaches approximately 0.99 fact accuracy on two independent holdouts and reduces critical false positives by 64–89%; the same data recipe also improves the Gemma backbone. This contribution addresses RQ2 and is described in Chapters 3 and 4, with evaluation in Chapter 6, Sections 6.1.2 and 6.1.3.
3. **A VR procedural training evaluation framework and formative pilot study.** The thesis delivers the instrumentation required to study evidence-grounded procedural tutoring with real participants: experiment export tooling, baseline and tutor-condition configurations, session launch support, replayable logs, semi-automated pre-scoring, error coding guides, and an experiment operation manual. A formative VR pilot study ($n = 9$) was conducted with four with-tutor and five without-tutor participants. All four tutor-condition participants completed the full 33-step procedure; none of the five without-tutor participants did. The observed error patterns also differed: tutor-condition errors were mainly assistance-coupled, while without-tutor errors were mainly omissions. These results support feasibility and instrumentation claims, but not controlled claims about learning effectiveness. This contribution addresses RQ3 and is documented in Chapter 6, Section 6.3, and Appendix 9.

Taken together, these contributions define the boundary of the thesis. The work demonstrates that evidence-grounded tutoring can be engineered, benchmarked, and

exercised in a formative VR study for a demanding simulator procedure. It does not claim that educational effectiveness has been established beyond that formative evidence.

1.6 Thesis Structure

Chapter 2 explains why the F/A-18C cold-start task requires multiple evidence sources, introduces the visual fact ontology, and positions the thesis against related work in intelligent tutoring, simulator instrumentation, state-aware assistance, VLM adaptation, and grounded AR/VR systems.

Chapter 3 presents the evidence-grounded architecture, including the ports-and-adapters structure, runtime data flow, procedure and gating model, telemetry processing, replay infrastructure, and VLM component.

Chapter 4 describes the methods used to create and validate the evidence chain: telemetry grounding, knowledge retrieval and source policy, visual fact data collection, ontology evolution, LoRA fine-tuning, and holdout benchmark design.

Chapter 5 documents the implementation choices that make the architecture operational, including core domain logic, adapter boundaries, structured help generation, overlay mapping, VLM integration, and experiment tooling.

Chapter 6 evaluates the thesis claims in two parts: technical evidence for RQ1 and RQ2, followed by the formative VR pilot evidence for RQ3.

Chapter 7 interprets the results, draws design lessons from the ontology revision, discusses scope boundaries, and states the limitations and future work.

Chapter 8 summarises the completed work against the research questions and re-states the claim boundaries of the thesis.

Appendix 9 contains supplementary figures, benchmark tables, dataset statistics, hyperparameter configurations, CLI references, and the 8-fact baseline benchmark.

2 Background and Related Work

The previous chapter framed the thesis around one requirement: generated procedural help should be grounded in evidence that can be inspected before and after it reaches the learner. This chapter explains why that requirement arises in the F/A-18C cold-start task and how it relates to existing work. The first part introduces the task, the available evidence channels, and the visual fact ontology used to turn cockpit screenshots into structured observations. The second part positions the thesis against intelligent tutoring systems, simulator-based training, rule-based state tracking, VLM adaptation, and grounded assistance in AR/VR.

2.1 The F/A-18C Cold-Start Task

The procedural scenario studied in this thesis is the F/A-18C cold-start in DCS World, a pre-flight cockpit procedure that moves the aircraft from a cold and dark state to a taxi-ready configuration [7, 8]. The task is useful as a research case because it is both bounded and state-sensitive. It has a known order of actions, yet successful completion depends on intermediate states that cannot be reduced to a text checklist: electrical power must be established before engine start, engine parameters must enter nominal ranges before the second engine is engaged, cockpit displays must show the expected pages, and some checks only make sense after earlier system transitions have occurred.

For a tutoring system, the relevant property of the cold-start is therefore not only its length. It is the way the procedure forces the tutor to combine three kinds of evidence: sequence evidence about which steps have been completed, telemetry evidence about cockpit controls and instruments, and visual evidence about display-page content. A response that ignores any of these signals can be locally fluent but procedurally wrong.

2.1.1 Procedure Structure: S01–S33 and Phases P1–P6

The current procedure pack represents the cold-start as 33 numbered steps (S01–S33) grouped into six phases. The phase structure is used in the thesis as a compact descrip-

tion of the evidence demands imposed by the task:

- P1 (S01–S03):** Power-up and safety checks. Battery and generator activation, fire-detection tests, and APU start establish the first electrical and safety preconditions for the rest of the procedure.
- P2 (S04–S07):** Right engine start. Right engine crank, throttle advance, bleed-air cycling, and caution/warning/advisory light testing introduce telemetry-dependent timing and verification.
- P3 (S08–S09):** Displays and communications. The DDIs, AMPCD, HUD, display pages, and COMM1 preset configuration make display state part of the procedural evidence chain.
- P4 (S10–S14):** Left engine and core systems. Left engine start, INS alignment mode selection, radar power, and OBOGS activation depend on earlier engine and avionics state.
- P5 (S15–S27):** FCS and flight controls. FCS reset and BIT, flap/trim configuration, launch-bar and arrestor-hook checks, refuelling probe checks, and pitot heat configuration mix telemetry-visible controls with display-based confirmation.
- P6 (S28–S33):** Pre-taxi instruments and final checks. Parking brake, BINGO fuel, altimeter settings, standby attitude indicator uncaging, and attitude-source selection complete the taxi-ready state.

Each step is represented in the procedure pack with precondition and completion gates. These gates make explicit which previous steps and which state variables must be satisfied before the tutor can treat a step as available or complete. For example, the right engine crank step depends on APU support, while the left engine start depends on right-engine parameters such as RPM, temperature, fuel flow, nozzle position, and oil pressure entering nominal ranges. This is why the task is a useful test case for evidence-grounded help: a tutor that advances the learner based on checklist order alone can miss the actual aircraft state.

2.1.2 Cockpit Displays and the Composite Panel

Many cold-start states are visible on the F/A-18C's cockpit displays rather than directly available as simple telemetry variables. The visual pipeline therefore operates on a fixed composite panel image containing three display regions:

- **Left DDI (Digital Display Indicator):** used during the procedure for pages such as FCS, SUPT, and TAC. In the evaluated cold-start flow, the FCS page and FCS page markings are important visual confirmations.
- **AMPCD (Advanced Multipurpose Color Display):** used for the HSI page, map layer, and INS alignment text. These states are central to the alignment-related visual facts.
- **Right DDI:** used for the BIT root page and FCS-MC BIT sequence, including intermediate, in-test, and final GO states.

Using a composite image keeps training and inference aligned: the model sees the same three-display layout during dataset construction, benchmarking, and runtime capture. More importantly, the composite panel defines a narrow perception boundary. The VLM is not asked to interpret the whole VR scene or infer the pilot’s intention. It only reports whether specific cockpit-display facts are visible in known regions.

2.1.3 DCS-BIOS Telemetry

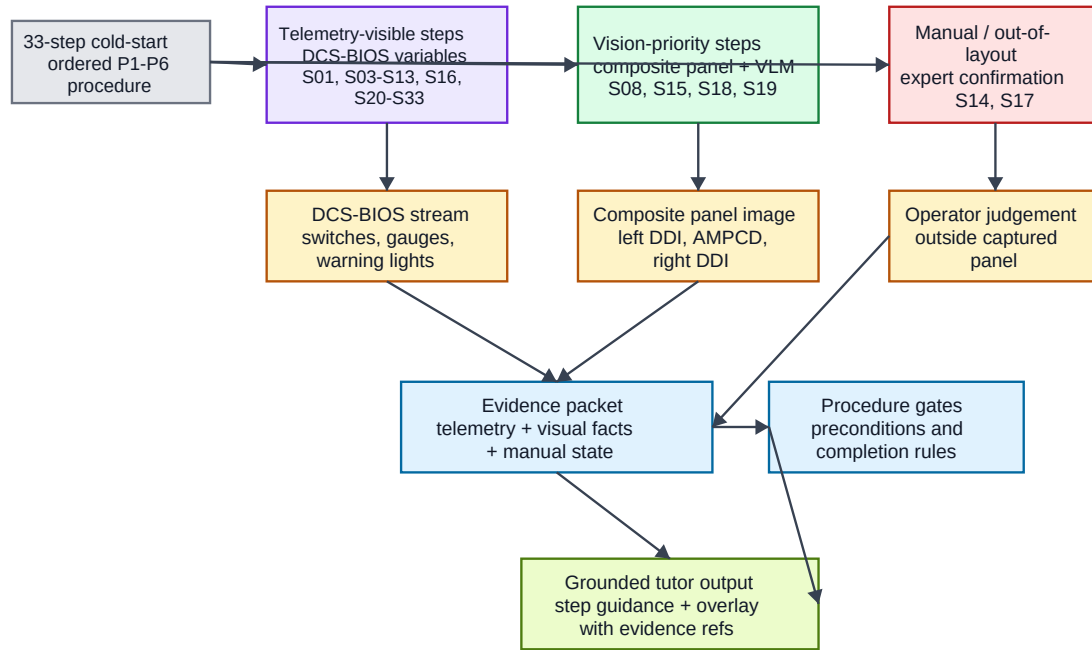
DCS-BIOS provides the main non-visual evidence source for the runtime. It exports cockpit switch positions, indicators, warning lights, gauges, and other instrument states from DCS World through a UDP telemetry stream [6]. The tutoring runtime normalises this stream into stable variables such as `rpm_r`, `apu_ready`, and `ins_mode`, applies delta sanitisation to reduce jitter, and tracks whether a source is missing rather than merely false.

This telemetry channel is authoritative where it exists. Steps such as battery power, throttle movement, radar mode, and many final pre-taxi switch states can be checked without computer vision. At the same time, DCS-BIOS does not export all display-page content needed by the cold-start tutor. The runtime therefore uses vision only for states that telemetry cannot observe, such as whether the FCS page, BIT root page, FCS-MC BIT result, or INS alignment text is visible. Figure 2.1 summarises this division of evidence across the 33-step procedure.

2.2 Visual Fact Ontology

The visual fact ontology is the contract between perception and procedural reasoning. Rather than asking a VLM to produce a tutor decision such as "the learner should press

F/A-18C cold-start evidence sources



The task cannot be grounded by language alone because completion evidence comes from mixed sources.

Figure 2.1: Evidence sources for the F/A-18C cold-start task. The 33-step procedure combines telemetry-visible steps, vision-priority display checks, and a small number of manual or out-of-layout confirmations; downstream tutoring decisions are grounded in the corresponding evidence source rather than in language-model output alone.

this button next", the runtime asks it to produce a small set of structured propositions about the visible cockpit display state. Each fact is a discrete observation with one of three values:

seen: the specified visual evidence is clearly present;

not_seen: the specified visual evidence is absent or not discernible;

uncertain: the image quality, occlusion, crop, or display state is insufficient for a confident decision.

This representation intentionally avoids mixing perception with task interpretation. The VLM reports whether visual evidence is visible. The procedure engine later combines those facts with telemetry, sequence state, and step bindings to decide whether a step is confirmed, pending, or blocked. The ontology is therefore not a minor implementation detail; it defines what kind of claim the model is allowed to make.

2.2.1 From 8-Fact v1 to 13-Fact v2

The ontology evolved through two generations. The first version contained eight facts over the left DDI, right DDI, and AMPCD. It included useful display-level targets such as FCS page visibility, BIT page visibility, FCS-MC page state, and INS alignment page visibility. It also included compound targets such as `ins_go`, which asked the model to infer a high-level procedural completion state from a single visual frame.

That design produced a clear lesson. The first LoRA adapter, trained on 180 human-reviewed Run-001 images, improved in-distribution behaviour but produced critical false positives on the independent Run-002 holdout: 13 of 50 samples were predicted as `seen` for `ins_go` when the human label was `not_seen`. The error was not only a model failure. It exposed a design problem in the target itself: `ins_go` combined visual perception with procedural interpretation.

The second-generation ontology replaces such compound targets with 13 observable visual atoms:

Region	Visual fact
Left DDI	tac_page_visible
Left DDI	supt_page_visible
Left DDI	fcs_page_visible
Left DDI	fcs_page_x_marks_visible
Right DDI	bit_root_page_visible
Right DDI	fcsmc_page_visible
Right DDI	fcsmc_intermediate_result_visible
Right DDI	fcsmc_in_test_visible
Right DDI	fcsmc_final_go_result_visible
AMPCD	hsi_page_visible
AMPCD	hsi_map_layer_visible
AMPCD	ins_grnd_alignment_text_visible
AMPCD	ins_ok_text_visible

The important change is not just the larger number of labels. The ontology moves from procedure-level judgement to observable evidence. For example, the v1 `ins_go` target is decomposed into AMPCD text and map-layer observations shown in the table above. The procedure engine then decides how these observations relate to the alignment step. This design principle carries the main scientific insight of the VLM part of the thesis: VLMs should extract local visual evidence, while procedural meaning should be assigned by explicit downstream rules.

2.2.2 Sticky Facts and Step Bindings

Two runtime mechanisms make the ontology usable during a live help cycle.

Sticky facts. Some visual events are transient but procedurally important. For example, the `fcsmc_final_go_result_visible` fact is marked as sticky with a time-to-live of 600 seconds. Once it has been observed as **seen**, later frames marked **not_seen** or **uncertain** do not immediately erase the evidence. This prevents a completed visual event from disappearing merely because the learner changed pages, the display flickered, or the capture window missed the relevant moment. Non-sticky facts, such as page visibility facts, expire quickly and are re-evaluated continuously.

Step bindings. Visual facts become procedurally meaningful only through explicit bindings in `vision_facts.yaml`. A binding specifies which facts must be visible for a step to be visually confirmed. Some bindings require all listed facts, such as S08 requiring both the FCS page on the left DDI and the BIT root page on the right DDI.

Others allow one of several alternatives when multiple visual cues can support the same procedural state.

The procedure pack also classifies each step by its preferred evidence source:

- `bios_priority_steps` for telemetry-visible states;
- `vision_priority_steps` for display-dependent states;
- `manual_or_out_of_layout_steps` for states requiring manual confirmation or lying outside the three-display composite.

This classification is the practical expression of the thesis’s evidence-grounding principle: each step is checked against the most reliable available signal before it is used in tutoring.

2.3 Intelligent Tutoring Systems and Procedural Training

Intelligent Tutoring Systems (ITS) provide the closest educational precedent for this work. Classic ITS research distinguishes expert knowledge, learner state, and tutoring feedback, and has shown that step-based computer tutors can be effective in structured learning domains [18]. In procedural settings, constraint-based and example-tracing tutors model expected actions and compare learner behaviour against authored task knowledge [2, 16]. These ideas directly inform the use of procedure packs and gates in this thesis.

The difference is the observation interface. Many ITSs operate inside a tutor-controlled environment where learner actions are already represented as formal events. The simulator used here is not a tutor-native interface. The runtime must infer procedure state from DCS-BIOS telemetry, cockpit screenshots, sequence history, and occasional manual confirmations. This makes evidence integration part of the tutoring problem itself.

LLMs add a further tension. They can produce fluent and flexible instructional dialogue, and recent educational discussions recognise their promise as well as their reliability risks [12]. For cockpit procedures, fluency is not enough. The contribution of this thesis is therefore not an open-ended LLM tutor, but a constrained tutoring runtime in which generated language is downstream of evidence collection, retrieval, procedure inference, and validation.

2.4 Simulator-Based Training and Instrumentation

Simulator-based aviation training is a mature area, and prior work has examined how simulation supports skill acquisition and transfer in flight-training contexts [10]. DCS World provides the high-fidelity simulation environment used in this thesis [7]. The relevant research issue for the present work is instrumentation: how the tutoring system observes what the learner and aircraft are doing without replacing the simulator with a custom tutor interface.

DCS-BIOS addresses part of this problem by exposing cockpit state through a structured telemetry bridge [6]. Compared with screen capture alone, telemetry gives deterministic access to many switch positions, indicator states, and numerical values. However, telemetry does not cover every piece of evidence that matters for the cold-start, especially display-page content and text. The thesis therefore treats simulator instrumentation as a multimodal problem: telemetry is used where it is reliable and direct, while vision is reserved for the cockpit evidence that telemetry does not export.

This separation also improves auditability. A logged help cycle can later show which state was read from DCS-BIOS, which facts came from the VLM, which rule made a step eligible or blocked, and which retrieved source supported the generated explanation. Instrumentation is thus not only an engineering requirement but a condition for evaluating the tutor’s behaviour.

2.5 Rule-Based and State-Aware Tutoring

The tutoring runtime combines deterministic state tracking with model-generated language. Rule-based approaches to procedure tracking, including finite-state, production-rule, and constraint-oriented models, offer interpretability because they make task structure explicit. The procedure pack in this thesis follows that principle: steps, preconditions, completion gates, visual bindings, and error categories are authored as declarative configuration rather than hidden in prompts.

The limitation of a purely rule-based tutor is that authored feedback can become brittle when the learner asks for help in an unexpected state or needs an explanation rather than a binary correction. The runtime therefore delegates natural-language formulation to an LLM, but not the authority to decide the procedure state. Step inference, eligibility, evidence availability, and overlay allowlists remain under deterministic control.

This hybrid division is related to broader work on grounding language models in action feasibility and symbolic structure, such as SayCan-style grounding of language in robotic affordances [1]. The present thesis adapts that grounding intuition to simulator tutoring: before a generated response can be acted upon, it must be tied to evidence that the runtime can observe, validate, and replay.

2.6 Foundation Models and VLMs for Situated Assistance

Foundation models provide two capabilities that are attractive for situated tutoring: language models can produce flexible explanations, and vision-language models can extract information from cockpit images. Parameter-efficient fine-tuning methods, especially Low-Rank Adaptation (LoRA), make it practical to adapt large models to narrow tasks with limited trainable parameters [11, 15]. The model backbones and VLM literature used in this thesis provide the general technical context for this adaptation setting [4, 5, 14].

The thesis uses these capabilities in a restricted way. The VLM is not asked for free-form cockpit descriptions, and the LLM is not asked to invent the next procedural state. Instead, the VLM outputs structured JSON visual facts, and the help-generation model receives an evidence packet assembled by the runtime. This choice follows the same general motivation as retrieval-augmented generation: model output should be conditioned on explicit external evidence rather than parametric memory alone [13].

The cockpit-display task also resembles structured screen understanding. Research on GUI and screen-content understanding has shown that visual models can reason about fixed regions, labels, and symbolic layout [19]. Cockpit displays share that structured quality, but the tutoring setting adds stricter consequences for false positives. A VLM that incorrectly reports a critical display state as visible may cause the tutor to advance the learner at the wrong time. This is why ontology design and critical-false-positive analysis are treated as central methodological issues in this thesis rather than as secondary benchmark details.

2.7 Explainable and Grounded AI Assistance in AR/VR

The original proposal placed the tutor in an immersive Quest 3 cockpit experience, and the implemented system retains that evaluation direction through the VR pilot. Prior work on AR guidance for maintenance and procedural tasks shows the value of presenting instructions in the user’s task context, often through spatially registered text, arrows, or animations [17]. Such systems demonstrate why in-context guidance is attractive: the learner can keep attention on the task environment rather than switching to external materials.

The system studied here differs from pre-authored AR guidance because its help content is generated at runtime from current evidence. That makes explanation and traceability central. Work on explainable AI and human-AI interaction emphasises that users and developers need to understand why a system produced a recommendation, especially in high-stakes or safety-sensitive settings [3, 9]. In this thesis, the explanation need is operationalised through evidence references: each recommended step, overlay target, or refusal to advance can be traced back to telemetry, a rule, a visual fact, or a retrieved source.

This grounding requirement also limits the claims made about VR. The pilot study in Chapter 6 shows that the runtime and instrumentation can be exercised with real participants in VR, but it does not establish controlled educational effectiveness. The VR contribution is therefore part of the evaluation framework and formative evidence, while the primary thesis claim remains the design and validation of the evidence-grounded tutoring runtime.

2.8 Positioning of This Work

The thesis sits at the intersection of the areas reviewed above, but its contribution is the particular integration rather than membership in any single area. From intelligent tutoring systems it takes the need for explicit task models and state-sensitive feedback. From simulator training it takes the need for instrumentation that observes real learner behaviour in a high-fidelity environment. From rule-based assistance it takes deterministic procedure gates. From foundation-model work it takes flexible language generation and domain-adapted visual extraction. From explainable and AR/VR assistance it takes the requirement that help remain inspectable in context.

The resulting system is an evidence-grounded procedural tutoring runtime. Its pro-

cedure pack defines what counts as valid state evidence; DCS-BIOS supplies telemetry where available; the visual fact ontology supplies structured cockpit display observations where telemetry is missing; retrieval supplies procedural sources; and the help cycle constrains model output through validation, allowlists, fallbacks, logging, and replay. This is the research position developed in the rest of the thesis.

The claim boundary is equally important. The technical chapters and benchmarks evaluate whether the architecture and visual fact pipeline can produce reliable, auditable evidence for tutoring. The formative VR pilot provides descriptive evidence about feasibility, completion, error patterns, and instrumentation with real participants. Larger controlled studies are still required before making claims about learning gains, workload reduction, retention, or transfer.

3 System Design

This chapter presents the architecture of a telemetry-grounded tutoring runtime for procedural training. It covers the design rationale, the evidence-grounded help cycle, the core components, and the harness engineering that enforces correctness. Methodology-level detail—the VLM adaptation protocol, dataset curation, and benchmark design—is deferred to Chapter 4; implementation-level specifics appear in Chapter 5.

3.1 Design Goals and Requirements

The thesis addresses the problem of providing grounded, context-aware procedural guidance in the F/A-18C cold-start procedure—a fixed sequence of 25 core steps (S01–S25, extended to S33 in the current pack) across six operational phases. Each step has preconditions that must be satisfied before execution and completion conditions that must be met before advancing. From this problem, six design goals are derived:

- G1 – Simulator-based procedural tutoring.** The system must observe learner actions through simulator state, determine the current procedural step, and provide timely, grounded guidance. Tutoring output must reference concrete cockpit elements and cite evidence.
- G2 – State-aware step guidance.** Procedure progress is tracked through a formal state machine. Preconditions are evaluated before step activation; completion conditions are verified before advancement. Guidance must be conditional on current simulator state, not merely step order.
- G3 – Separation of tutoring logic from simulator integration.** The core tutoring logic—procedure tracking, gating, scoring, help generation—must be independent of any particular simulator or model provider. Simulator-specific code is isolated behind stable interfaces so the core can be tested without a running simulator.

- G4 – Replayability and inspectability.** Every interaction is recorded as structured, timestamped events supporting offline replay, automated scoring, and post-hoc analysis.
- G5 – Model-optional operation with deterministic fallback.** When a language model is available, it provides richer diagnoses; when unavailable or when its output fails validation, the system degrades gracefully to deterministic, rule-based behaviour. A VLM-based visual fact extraction path complements telemetry gating for display-page states not exported through DCS-BIOS.
- G6 – Declarative procedure configuration.** Procedure definitions, UI targets, telemetry mappings, error taxonomies, and vision fact ontologies are expressed as declarative YAML configuration (a *procedure pack*) rather than hard-coded in source code.

3.2 Architecture: Ports and Adapters

The system follows a ports-and-adapters (hexagonal) architecture organised into three layers with strict dependency direction: **adapters** $\rightarrow$ **ports** $\leftarrow$ **core**. The core never references adapter code directly.

Core layer. Contains all domain logic independent of any simulator, model provider, or external service: the procedure state machine, gating rule engine, scoring and interaction-metrics engines, vision fact ontology with sticky/TTL persistence, BM25 retrieval, dynamic JSON Schema generation for help responses, and security infrastructure.

Ports layer. Defines four stable Python protocols: **ModelPort** (reasoning), **KnowledgePort** (retrieval), **VisionPort** (visual input), and **TelemetryPort** (simulator state). These are the only dependency injection points between core and the outside world.

Adapters layer. Contains all simulator-specific, provider-specific, and infrastructure-specific code: DCS-BIOS telemetry receivers, telemetry enrichment and delta sanitisation, DCS overlay senders, OpenAI-compatible model clients (with and without multimodal support), Ollama fallback, a deterministic model stub, local BM25 knowledge adapter, and the vision fact extraction runtime.

Two further structural elements complete the architecture:

Procedure packs (packs/). A directory of declarative YAML files describing one procedural task: step list, precondition and completion gates, UI targets and element identifiers, telemetry variable mappings, error taxonomy, delta sanitisation policy, and vision fact ontology with step bindings. The current repository contains the `fa18c_startup` pack.

Runtime entrypoints. The `simtutor` package provides a command-line interface for simulation, replay, scoring, VLM recording, and validation. The live runtime module orchestrates the full loop: DCS-BIOS ingestion, telemetry enrichment, knowledge retrieval, model-based help generation, VLM visual fact extraction, overlay execution, and event logging.

Figure 3.1 summarises the thesis-level architecture. The figure abstracts away most file-level implementation details and emphasises the evidence flow between simulator state, procedure-pack configuration, foundation-model services, harness validation, and study outputs.

The separation serves four purposes. **Testability:** core logic can be tested with mock adapters without DCS, a GPU, or network access. **Maintainability:** simulator API changes affect only the relevant adapter. **Transferability:** porting to a new simulator requires new adapters and packs; the core is reused as-is. **Independent evolution:** the VLM pipeline can be developed, trained, and benchmarked independently of the tutoring runtime.

3.3 The Evidence-Grounded Help Cycle

The central architectural mechanism is the evidence-grounded help cycle: a structured pipeline from simulator observation through tutoring output and logging, in which every actionable output must carry a traceable evidence reference. Figure 3.2 shows this cycle from help trigger to overlay dispatch and event logging.

3.3.1 Help Cycle Stages

Observation. The simulator adapter—a DCS-BIOS UDP receiver—produces a stream of timestamped observations containing raw simulator state. These are enriched through variable resolution and delta processing into normalised key-value state variables.

Tutor request. When the learner triggers help, a tutor request is created carrying the current observation and an assembled context: normalised state variables, sanitised

Evidence-grounded tutoring architecture

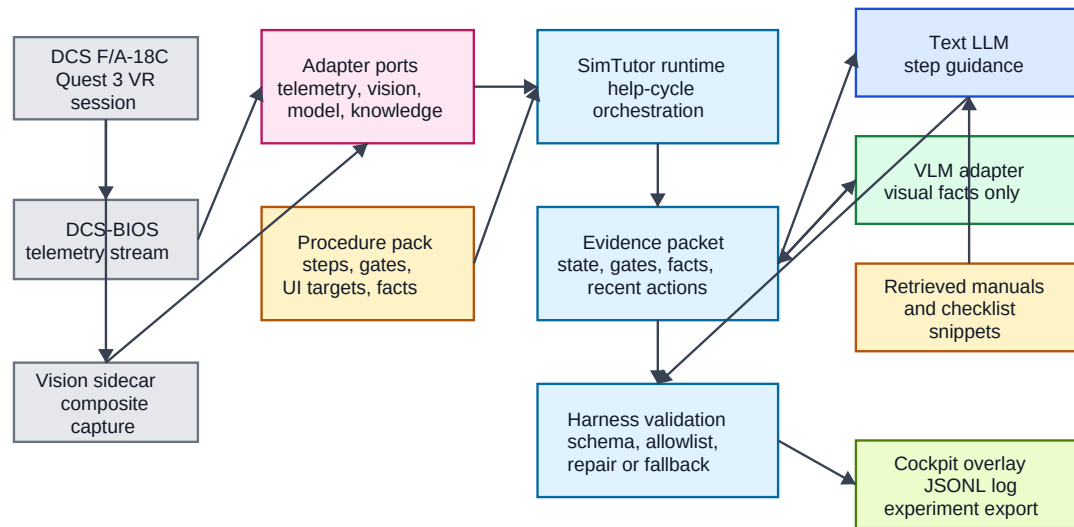
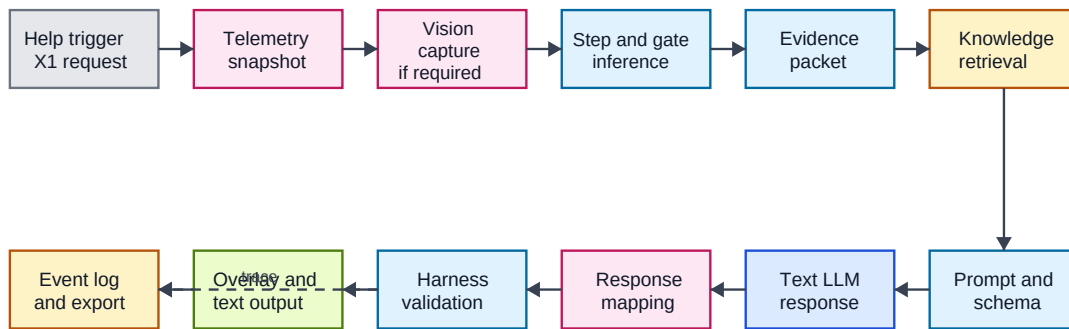


Figure 3.1: Evidence-grounded tutoring architecture. Simulator telemetry, composite-panel vision, procedure-pack configuration, retrieved knowledge, and model outputs are integrated through an evidence packet and checked by the harness before any overlay or tutor text reaches the cockpit.

Live evidence-grounded help cycle



The LLM never acts directly on the simulator; mapped responses pass through harness validation first.

Figure 3.2: Live evidence-grounded help cycle. Each help request is converted into a telemetry and vision observation, interpreted against procedure gates, grounded with retrieved knowledge, passed through the text LLM, validated by the harness, and then dispatched as overlay/text output with an event-log trace.

recent deltas, candidate procedure steps, a deterministic step hint, an overlay target allowlist, BM25-retrieved knowledge snippets ($k = 5$), and, when available, the current vision fact summary.

Model inference. The assembled context is formatted into a prompt and sent to the language model, which returns a structured JSON object conforming to a dynamically generated HelpResponse JSON Schema.

Response validation and mapping. The raw model output passes through four validation stages: JSON extraction with minimal repair (code fence removal only), schema validation against the runtime-generated schema, allowlist checks on step identifiers and overlay targets, and evidence-grounding validation. Validated responses are mapped to overlay actions resolved through the UI map. Failed validation triggers deterministic fallback.

Action execution. Overlay actions (highlight, clear, pulse) are validated against the UI target allowlist and dispatched to the simulator. Auto-click and control-injection are forbidden.

Event logging. Every stage is recorded as a versioned, timestamped event in a JSONL event log—the authoritative record for all downstream analysis, replay, and scoring.

3.3.2 Schema-Level Evidence Grounding

The evidence constraint operates at the schema level, not as a post-hoc check. The HelpResponse JSON Schema is generated dynamically from the procedure pack and UI map at runtime. For every overlay target in the schema’s `targets` array, a conditional `if/then` rule requires that the accompanying `evidence` array contain at least one item referencing that target. Each evidence item must declare a `type` from five grounding kinds—telemetry variable (`var`), gating rule (`gate`), knowledge snippet (`rag`), recent state change (`delta`), or vision fact (`visual`)—each with a corresponding `ref` prefix. An LLM response that proposes an overlay without a matching, validly-typed evidence item is structurally invalid and rejected before any action reaches the simulator.

This mechanism produces a unified audit trail. For example, highlighting the `battery_switch` must be accompanied by an evidence item such as `{"target": "battery_switch", "type": "var", "ref": "VARS.battery_switch", "quote": "0"}`, tracing the action to the telemetry variable that justified it. The five evidence types are validated against the keys actually present in the help-cycle context; fabricated references are rejected.

3.4 Telemetry Processing

The telemetry processing subsystem transforms raw DCS-BIOS data into stable state representation through four stages:

1. **Raw ingestion.** The DCS-BIOS receiver decodes binary UDP export frames into structured JSON with wall-clock timestamps and monotonic sequence numbers.
2. **Variable resolution.** A telemetry map (declared in the procedure pack) projects raw BIOS fields onto normalised state variables. The resolver tracks unavailable sources in a `vars_source_missing` set, enabling downstream components to distinguish “value is zero” from “value is unknown”—a distinction critical for correct gating behaviour.
3. **Delta sanitisation.** A configurable policy (per-variable thresholds, debounce windows, filtering rules) suppresses spurious changes from switch bounce, sensor drift, and UDP out-of-order delivery.
4. **Delta aggregation.** Sanitised deltas are accumulated over a configurable window and compressed into a compact prompt-safe summary.

The telemetry pipeline runs continuously, independent of any help cycle.

3.5 Procedure Engine and Gating Rules

3.5.1 Procedure State Machine

The procedure engine maintains a finite-state machine over the ordered step sequence. Each step is in one of four states: `pending`, `active` (at most one), `done`, or `blocked`. All transitions emit internal event records with ISO-8601 timestamps.

3.5.2 Gating Rule Engine

Precondition and completion gates are evaluated using a DSL with four operators: `var_gte` (threshold), `arg_in_range` (interval), `flag_true` (boolean), and `time_since` (elapsed time). The evaluator includes explicit unknown-value handling: missing, source-missing, or sentinel-value variables cause rule failure with a diagnostic reason. Rules short-circuit on first failure; the failure index and reason feed into the deterministic step hint.

3.5.3 Deterministic Fallback

When no model is available or model output fails validation, the system actively computes a useful step hint through deterministic inference. The fallback analyses three information sources: (1) the active step and its unsatisfied gating rules, (2) the learner’s recent UI interactions, and (3) observability metadata that classifies each missing evidence source as a *hard blocker* (gate explicitly fails) or *soft blocker* (telemetry source absent, true state unknown). This distinction prevents the tutor from incorrectly claiming a step is unsatisfied when the evidence is simply unavailable. The fallback produces a text-only hint restricted to the active step; overlay highlights are suppressed unless a local rule can uniquely determine a valid target.

3.5.4 Step Classification by Evidence Source

Each step is classified by its primary evidence source, declared in the procedure pack:

- **BIOS-priority steps** (S01, S03–S07, S09–S13, S21): verified primarily through DCS-BIOS telemetry.
- **Vision-priority steps** (S02, S08, S14–S20, S22–S25): verified through VLM visual fact extraction from cockpit displays.
- **Manual/out-of-layout steps**: steps requiring manual confirmation or involving cockpit areas outside the three-display composite panel.

This tripartite classification ensures each step’s completion evidence is sourced from the most reliable available signal.

3.6 Harness Engineering and Evidence Validation

A distinguishing architectural feature of the system is the *step harness*: a validation layer that sits between the help-cycle response generation and the overlay action execution, enforcing that every actionable output satisfies evidence-grounding, allowlist, and procedural consistency constraints before reaching the simulator.

3.6.1 Harness Architecture

The harness operates in four stages applied sequentially to every candidate tutor response:

1. **Spec derivation.** For each active procedure step, the harness derives a *step harness spec* from the procedure pack: the set of permitted overlay targets (from the UI map), the required evidence categories (from the step’s gating configuration and observability metadata), and the banned target categories (e.g., targets belonging to a different procedure phase or requiring visual confirmation when the vision path is unavailable).
2. **Decision adjudication.** The harness evaluates the model’s proposed diagnosis and next-step recommendations against the current procedure state. A `HarnessDecision` dataclass records whether each proposed action is **accepted**, **repaired** (modified to align with procedure state), or **rejected** (blocked as unsafe or inconsistent). Repairs are traceable: every modification carries a reason code referencing the specific harness rule that triggered it.
3. **Evidence snapshot trace.** Before any overlay action is executed, the harness captures an *evidence snapshot*—a frozen record of the telemetry variables, gating rule evaluations, knowledge snippet references, and vision fact states that jointly justify the current overlay plan. This snapshot is embedded in the event log alongside the tutor response, creating a verifiable link between the evidence that was available at decision time and the action that was taken.
4. **Coverage matrix.** A regression coverage matrix maps each procedure step to a set of harness test scenarios (fixture-based replay cases), each exercising a specific harness rule: correct advancement, safe rejection of wrong-target proposals, repair of incomplete visual evidence, handling of moving/settling controls, and prevention of VLM evidence leakage into non-visual steps.

The detailed validation diagram is retained in Appendix Figure 9.1. For the main thesis argument, the key point is that the harness converts a proposed tutor action into an auditable decision and falls back to repair or text-only guidance when overlay execution is not sufficiently grounded.

3.6.2 Fixture-Based Regression Testing

The harness is checked through a suite of *live help fixtures*: recorded telemetry snapshots captured at specific procedure states that previously exposed harness failures. Each fixture replays a specific state through the full help cycle and asserts that the harness

produces the expected outcome (correct target, correct step, or correct rejection). As of the current implementation, the regression suite covers 20 live-fixture regression entries in the coverage matrix spanning steps S01 through S33, including:

- Correct advancement when completion conditions are satisfied (telemetry-based and vision-based);
- Safe rejection of overlay targets that violate the UI allowlist or belong to a different procedure phase;
- Repair of wrong-target proposals (e.g., PB18 repaired to PB5 for BIT root sub-state);
- Prevention of VLM evidence leakage into non-visual steps (stale S08 visual facts must not drive S09/S10 help);
- Correct handling of moving/settling controls (refuelling probe, arresting hook) that require wait guidance rather than repeated prompts;
- Consistent interaction semantics (left-click vs. right-click, mouse wheel for rotary controls).

The harness coverage matrix and fixture suite serve a dual role: they are both a development-time regression safety net and an existence proof that systematic evidence-grounding validation can be enforced at the architectural level rather than relying on model output trustworthiness.

3.7 Simulator Adapter Layer

The simulator adapter layer isolates DCS-specific code behind the port interfaces. Three adapters are defined:

Telemetry input. Receives DCS-BIOS state data over UDP and feeds the telemetry enrichment pipeline.

Overlay output. Sends highlight, clear, and pulse commands to DCS via UDP. A Lua script inside DCS renders graphical overlays on cockpit elements. The adapter maps abstract target names to DCS-internal point codes using the pack's UI map.

Vision input. Triggers screenshot captures in DCS. The Lua-side integration renders the three primary cockpit displays into a single composite image; the Python side manages frame selection and passes images to the VLM extractor.

The VR code path has been implemented (VR_MIRROR configuration, tutor text UDP injection, composite-panel capture compatibility in VR mode) and was exercised in a formative pilot study (Section 6.3); larger controlled human-subject experiments remain future work.

3.8 Event Logging and Replay

All runtime occurrences are represented as versioned, timestamped **Event** objects with a fixed schema and persisted in JSONL format via an append-only store with immediate flush after each write, ensuring crash-safe logging.

A recorded event log can be replayed offline to verify step ordering and state machine consistency. The replay evaluation infrastructure supports suite-driven regression replay: a YAML suite file specifies a pack path, test cases with expected outcomes, and optional vision frame directories. Five replay evaluation cases cover distinct scenario types (no-operation, battery-on, battery-and-generator-off, FCS reset and BIT, INS alignment), each with DCS-BIOS raw captures and per-frame sidecar files.

The scoring engine counts omissions and state violations with configurable per-category weights. Interaction metrics characterise learner behaviour: stall time, help requests, LLM triggers, UI click counts, and overlay request/acknowledgement counts.

3.8.1 Current Limitations

The replay infrastructure has been exercised with synthetic regression scenarios and fixture-based replay at the telemetry level, including cases with vision frame sidecars. End-to-end replay of full multimodal sessions with synchronised vision frames from live human-in-the-loop runs has not yet been validated.

3.9 VLM Adaptation for Visual State Understanding

The VLM component extracts structured *visual facts* from cockpit screenshots—intermediate propositions about visible display content. Visual facts are not final tutoring decisions; they provide visual evidence that the tutoring runtime combines with telemetry, procedure state, and documentation. This layered architecture isolates the VLM’s responsibility to structured visual evidence extraction, reserving procedure-level decisions for deterministic rule-based components.

3.9.1 Vision Fact Ontology (13-Fact v2)

The current ontology defines 13 facts spanning three display regions:

- **Left DDI:** `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `fcs_page_x_marks_v`
- **Right DDI:** `bit_root_page_visible`, `fcsmc_page_visible`, `fcsmc_intermediate_result_v`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`.
- **AMPCD:** `hsi_page_visible`, `hsi_map_layer_visible`, `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`.

Each fact has a three-valued state (`seen`, `not_seen`, `uncertain`), a time-to-live, an optional `sticky` flag (once `seen`, not overwritten by subsequent `not_seen` within TTL), and step bindings via `all_of` / `any_of` conditions. The evolution from the 8-fact v1 ontology to the current 13-fact v2 design is a central methodological and evaluation narrative, detailed in Chapter 4 and evaluated in Chapter 6.

3.9.2 VLM Adaptation Pipeline

The VLM used at runtime is fine-tuned through a seven-stage pipeline spanning screenshot capture, AI pre-labeling, human review in Label Studio, bilingual supervised fine-tuning export, LoRA fine-tuning via Unsloth + PEFT + TRL, and automated base-vs-LoRA benchmarking on independent holdout sets with verified zero training overlap. Full methodological detail appears in Chapter 4.

3.9.3 Runtime Integration

When a help request arrives and the active step is vision-priority, a capture trigger is sent to DCS. Pre-trigger and trigger frames are rendered and sent to the VLM. The returned facts are merged into the vision fact snapshot (with sticky and TTL semantics) and included in the help-cycle context. If the vision adapter is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only. This graceful degradation is a deliberate design property: visual perception is an optional enhancement to the telemetry-grounded core, not a dependency.

4 Methodology

This chapter describes the methodological framework for the telemetry-grounded tutoring runtime and the VLM adaptation pipeline. Section 4.1 covers telemetry processing and gating rule evaluation. Section 4.2 describes knowledge grounding via BM25 retrieval with source policy. Section 4.3 presents the seven-stage visual fact data pipeline. Section 4.4 details dataset curation and the ontology evolution from 8-fact v1 to 13-fact v2. Section 4.5 specifies the LoRA fine-tuning configuration for three backbone models under matched data recipe. Section 4.6 defines the benchmark protocol, metrics, and holdout strategy.

4.1 Procedure Runtime and Telemetry Grounding

The procedure runtime ingests raw simulator telemetry, normalises it into stable state variables, evaluates gating rules to determine step eligibility, and provides the resulting state representation to the help-cycle context assembler.

4.1.1 Variable Resolution and Source-Missing Tracking

A telemetry map (declared in the procedure pack) defines how raw DCS-BIOS fields are projected onto normalised state variables. The variable resolver produces a flat dictionary of key–value pairs and tracks unavailable or unknown sources in a `vars_source_missing` set, enabling downstream components to distinguish “the value is zero” from “the value is unknown”—a distinction essential for correct gating rule behaviour. A missing variable never silently evaluates to a successful condition.

4.1.2 Gating Rule Evaluation

The procedure engine evaluates two gate types per step: precondition gates (before activation) and completion gates (before marking done). The DSL supports four operators: `var_gte` (threshold), `arg_in_range` (interval), `flag_true` (boolean), and `time_since`

(elapsed time). The evaluator includes explicit unknown-value handling: missing, source-missing, or sentinel-value variables cause rule failure with a diagnostic reason. Rules short-circuit on first failure; the failure index and reason propagate to the deterministic step hint and help-cycle context.

4.1.3 Delta Sanitisation and Aggregation

Raw telemetry variables change at high frequency due to simulator physics even without learner action. Two processing stages produce a stable representation: delta sanitisation suppresses spurious changes via per-variable thresholds, debounce windows, and filtering rules; delta aggregation accumulates sanitised deltas over a configurable window and compresses them into a prompt-safe summary included in the help-cycle context.

4.1.4 Deterministic Fallback

When no language model is available or model output fails validation, the procedure runtime falls back to deterministic step inference. The fallback examines the active step, its unsatisfied gating rules, and recent UI targets to produce a text hint restricted to the active step. Overlay highlights are suppressed unless a local rule can prove a unique, valid target. The mechanism distinguishes hard blockers (gate explicitly fails) from soft blockers (telemetry source absent), preventing incorrect claims that a step is unsatisfied when evidence is simply unavailable.

4.2 Knowledge Grounding and Source Policy

Knowledge retrieval uses BM25, a bag-of-words ranking function from the probabilistic information retrieval family. Documents are indexed at the sentence level from the F/A-18C NATOPS flight manual and supplemental cold-start documentation. At retrieval time, a grounding query is constructed from the active step identifier, its description, and the current telemetry observation. The query is scored against indexed sentences using standard BM25 weighting ($k_1 = 1.2$, $b = 0.75$); the top- k scoring snippets ($k = 5$) are included as evidence context in the help-cycle prompt.

A declarative source policy enforces an allowlist of permitted document sources. Each indexed sentence carries a source identifier; only snippets from allowed sources are included in retrieval results. This prevents the model from being exposed to procedure variants that contradict the current pack configuration (e.g., carrier-based procedures

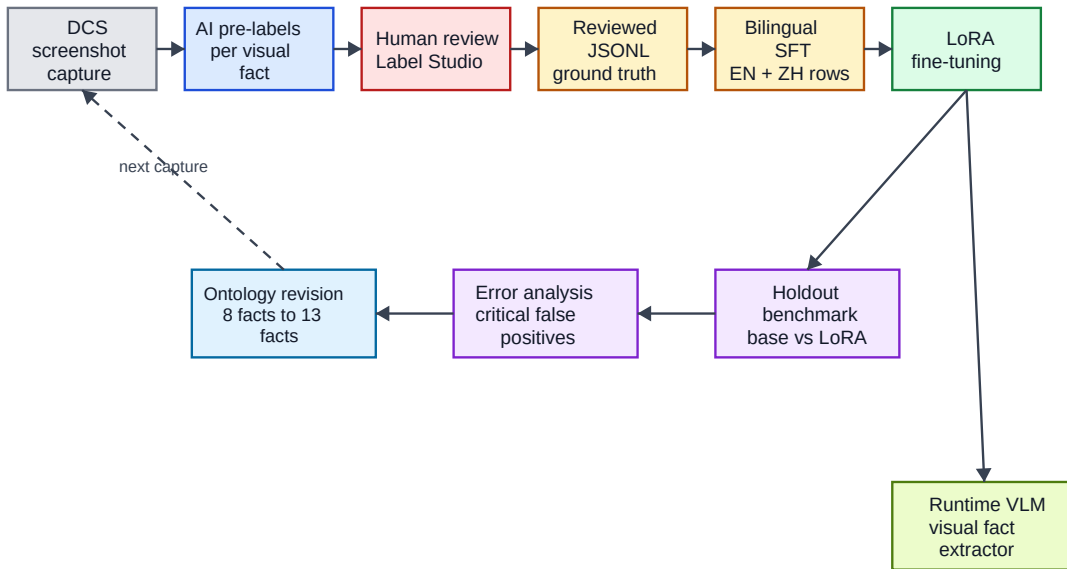
when the current profile is airfield) and bounds the retrieval scope to curated, verified documentation.

4.3 Visual Fact Data Pipeline

The visual fact data pipeline transforms raw DCS cockpit screenshots into structured visual fact extraction models through seven stages. Each stage produces versioned, inspectable artifacts, enabling full traceability from raw image to benchmark metric.

Figure 4.1 shows the pipeline as a methodological loop: holdout benchmark failures inform ontology revision and later data collection rather than being treated only as final scores.

Small-data VLM adaptation pipeline



Each benchmark prediction remains traceable to a reviewed label and original cockpit image.

Figure 4.1: VLM adaptation pipeline. DCS composite screenshots are pre-labelled, human-reviewed, converted into bilingual supervised fine-tuning rows, adapted with LoRA, evaluated on independent holdout sets, and used to drive ontology revision.

1. **Screenshot capture.** DCS renders the three primary multifunction displays into a single composite-panel image. Captures are triggered at configurable intervals

and stored alongside a sidecar manifest. The composite format keeps training, pre-labeling, evaluation, and runtime inference on the same input distribution.

2. **VLM pre-labeling.** A large VLM (Qwen 397B-class, accessed via OpenAI-compatible API) produces initial per-fact labels and evidence notes for each image. Pre-labeling seeds annotations at scale; human review corrects errors in the next stage.
3. **Human review in Label Studio.** Pre-labeled samples are imported into Label Studio. Each task presents the composite image, predicted labels, and a summary field. The reviewer corrects mislabeled facts and provides evidence notes. All reviewed samples carry an audit trail linking back to the raw image, the original pre-label, and the reviewer’s corrections.
4. **Reviewed JSONL export.** Label Studio exports corrected labels as a JSONL file—the authoritative ground-truth artifact for all downstream stages.
5. **Bilingual SFT JSONL generation.** Each reviewed sample produces two training rows—one English, one Chinese—sharing the same image and fact labels but differing in instruction text. Each row is a three-message conversation: system message (JSON-only instruction), user message (image + fact definitions + decision boundaries), and assistant message (ground-truth JSON with `facts` array and `summary`). External metadata fields (`frame_id`, `session_id`) are excluded from the training target.
6. **LoRA fine-tuning.** The SFT JSONL files are used to fine-tune a base VLM via Low-Rank Adaptation (LoRA) using Unsloth (4-bit loading, LoRA injection, vision batch collation), PEFT (adapter definition), and TRL `SFTTrainer` (supervised fine-tuning loop). Adapter weights are saved separately from the base model.
7. **Base-vs-LoRA benchmark.** The fine-tuned LoRA adapter is evaluated against the unadapted base model on independent holdout sets. The benchmark script loads each model configuration in 4-bit inference mode, sends each sample through the same prompt used during training, validates output against the expected JSON schema, and computes metrics comparing predictions to human-reviewed ground-truth labels.

A defining methodological property is end-to-end traceability: every benchmark prediction can be traced back through training data, human-reviewed labels, VLM pre-

labels, and the original DCS screenshot. The pipeline produces versioned artifacts at each stage, enabling rigorous audit from individual fact misclassification to aggregate benchmark metrics.

4.4 Dataset Curation

4.4.1 Dataset Overview

The visual fact data pipeline produced six datasets over successive capture and review cycles. Table 4.1 summarises each dataset.

Table 4.1: Dataset inventory. All counts from `stats.json` files.

Dataset	Role	Samples	Ontology	Review
Run-001 (<code>vision_sft</code>)	Training source	180	8-fact v1	180
Run-002 (<code>vision_sft_holdout_run002</code>)	Holdout	50	8-fact v1	50
Run-002 newfacts	Holdout	50	13-fact v2	50
Run-003 (<code>vision_sft_run003</code>)	Training source	220	13-fact v2	220
Run-004 (<code>vision_sft_holdout_run004_random</code>)	Holdout	100	13-fact v2	100
Run-005 (<code>composition_rebalance</code>)	Training supplement	122	13-fact v2	122

Run-003 and Run-005 contain samples where the human reviewer left the summary field blank (82 and 25 samples, respectively). These samples remain valid for training and evaluation because the facts array—the primary supervised target—is fully annotated for every fact in every sample.

4.4.2 Ontology Evolution: 8-Fact v1 to 13-Fact v2

The initial 8-fact v1 ontology defined eight facts spanning three display regions. Two structural weaknesses were identified during early benchmark analysis. First, it mixed abstraction levels: some facts described low-level visual observations (`fcs_page_visible`), while others encoded higher-level procedural judgements (`ins_go`, representing an INS alignment completion signal rather than a directly observable visual feature). Second, several facts shared overlapping semantics: `right_ddi_fcsmc_page_visible` conflated region identifiers with fact semantics, and the compound `ins_go` fact asked a single-frame VLM to make a procedural completion judgement that conflated perception with reasoning.

Figure 4.2 illustrates the central ontology change. Compound procedural targets were split into visual atoms, leaving temporal and procedural interpretation to the tutor runtime.

Visual fact ontology evolution

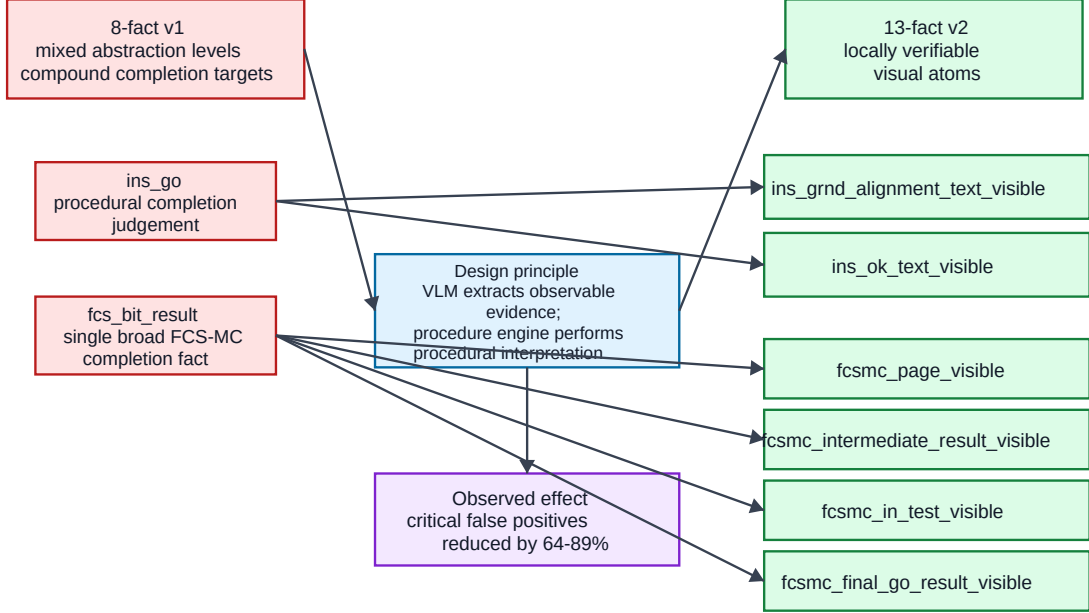


Figure 4.2: Ontology evolution from 8-fact v1 to 13-fact v2. Compound targets such as `ins_go` and broad FCS-MC completion facts were decomposed into locally verifiable visual atoms; the procedure engine then combines those observations with telemetry, phase, and timing evidence.

The 13-fact v2 ontology, used in Run-003 through Run-005 and in the current runtime, addressed these weaknesses through three strategies:

1. **Decomposition of compound targets.** The ambiguous `ins_go` was decomposed into two lower-level, directly observable facts: `ins_grnd_alignment_text_visible` (requiring literal GRND alignment block text) and `ins_ok_text_visible` (requiring clearly readable OK text near the alignment block). The FCS-MC state sequence was decomposed from a single `fcs_bit_result_visible` into a four-state chain (`fcsmc_page_visible`, `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`).

2. **Region-independent naming.** Region identifiers were removed from fact names (`right_ddi_fcsmc_page_visible` became `fcsmc_page_visible`); page-visibility facts were added for TAC and SUPT menus to cover the left DDI; HSI-related facts were split into `hsi_page_visible` and `hsi_map_layer_visible`.
3. **Abstraction-level discipline.** Every v2 fact is a locally verifiable visual proposition evaluable from a single cockpit image without procedural context. The downstream procedure engine—not the VLM—combines atomic observations into procedural judgements using telemetry, phase, and timing information.

4.4.3 Training Recipe: Run-003 + Run-005 $\times$ 2

The primary training recipe combines Run-003 (220 reviewed samples, bilingual export: 220 EN + 220 ZH = 440 rows) with Run-005 composition rebalance data included twice (122 reviewed samples, bilingual export: $122 \times 2 \times 2 = 488$ rows), yielding a combined training set of **928 rows**.

Run-005 was captured specifically to address class imbalance in Run-003, where several critical facts had extremely low **seen** counts: `ins_ok_text_visible` (12 of 220), `fcs_page_x_marks_visible` (16), and the three FCS-MC sub-states (18 each). Run-005 targeted cockpit states in which these rare facts are **seen**; double-counting (Run-005 $\times$ 2) further amplifies their representation. No internal evaluation split was used (`eval_rows` = 0); all evaluation is performed on independent holdout datasets.

4.4.4 Holdout Independence

Two holdout sets were constructed for the 13-fact v2 evaluation: Run-002 newfacts (50 samples, re-annotated from the Run-002 capture session) and Run-004 random (100 samples, uniformly random cockpit state sampling). Both are distinct DCS capture sessions with verified zero exact overlap with any training data—the overlap check compared image artifact paths and sample identifiers across training and holdout splits.

In contrast, the Run-001 benchmark used in early 8-fact v1 experiments is a **contaminated development set**: the evaluation data pool overlaps with the training data pool. Run-001 results are reported for regression analysis only (see Appendix 9) and are explicitly distinguished from independent holdout results.

4.5 LoRA Fine-Tuning Configuration

4.5.1 Backbone Models and Comparison Strategy

Three VLM backbones were fine-tuned under the matched Run-003 + Run-005 $\times$ 2 data recipe (928 bilingual rows, 13-fact v2 ontology), organised in three comparison tiers:

Primary — Qwen3.5-9B. A 9-billion-parameter VLM (Qwen/Qwen3.5-9B-Base). This is the primary experimental backbone, selected for its balance of capability and training efficiency. LoRA adapters are applied to both vision and language layers.

Scale/version comparison — Qwen3.6-27B. A 27-billion-parameter VLM from the same model family (Qwen/Qwen3.6-27B). This comparison examines the combined scale/version effect within the Qwen family under a matched data recipe and holdout protocol. The base model achieves substantially higher base-model performance (fact accuracy 0.91 on Run-002 newfacts, vs. 0.87 for Qwen3.5-9B), but the LoRA improvement margin is narrower (fact accuracy $\Delta = +0.077$ vs. $+0.123$ for Qwen3.5-9B), consistent with the expectation that larger base models have less adaptation headroom under small-data fine-tuning.

Cross-family — Gemma-4-31B. A 31-billion-parameter VLM from Google DeepMind (google/gemma-4-31B). This comparison tests whether the data recipe generalises across model families. LoRA is restricted to language-only linear modules (`all-linear`; vision layers are not fine-tuned).

4.5.2 Training Stack and Hyperparameters

All three backbones share the same training stack: Unsloth (4-bit model loading, LoRA injection, vision batch collation), PEFT (LoRA adapter definition), and TRL `SFTTrainer` (supervised fine-tuning loop). Hyperparameters are matched wherever architecturally possible: learning rate 2×10^{-4} , LoRA rank $r = 16$, $\alpha = 16$, dropout = 0.0, 4 epochs, effective batch size 4 (batch size 1 $\times$ gradient accumulation 4), maximum sequence length 4096, seed 3407, warmup ratio 0.1, weight decay 0.01. Backbone-specific differences—LoRA target modules, vision layer fine-tuning, chat template, and GPU memory utilisation—are documented in Appendix 9, Table 9.1.

The epoch count of 4 was selected as a compromise for small-data domain adaptation: with 928 training examples, fewer epochs may not reliably teach the fixed JSON schema and cockpit fact boundaries, while more epochs risk overfitting to the training sessions.

LoRA rank $r = 16$ provides moderate adaptation capacity—sufficient for cockpit layout patterns and JSON formatting without the overfitting risk of high-capacity adapters ($r \geq 64$). Dropout was set to zero for these first adaptation experiments, prioritising learnability over regularisation.

4.5.3 Bilingual SFT and Composition Rebalance

The bilingual SFT strategy produces two training rows per reviewed sample (English and Chinese instructions, identical image and ground-truth labels). This increases instruction-following robustness without additional annotation effort, but does not substitute for additional cockpit-state screenshots or hard-negative visual data.

The composition rebalance strategy (Run-005) addresses the class imbalance observed in Run-003. Several critical facts had extremely low positive counts: `ins_ok_text_visible` (12 of 220, 5.5%), `fcs_page_x_marks_visible` (16, 7.3%), and the three FCS-MC sub-states (18 each, 8.2%). Run-005 captured 122 additional samples targeting cockpit states where these rare facts are **seen**, improving combined coverage substantially (e.g., `supt_page_visible` from 20 to 54 combined seen; `hsi_page_visible` from 106 to 215; `ins_grnd_alignment_text_visible` from 58 to 130). Double-counting Run-005 in the recipe further amplifies the signal.

4.6 Benchmark Protocol

4.6.1 Metrics

The benchmark compares model predictions against human-reviewed ground-truth labels using the following metrics:

json_valid_rate Proportion of samples with parseable JSON output. Non-JSON responses (markdown, commentary, malformed braces) count as failures.

schema_valid_rate Proportion of samples whose parsed JSON conforms to the expected schema (correct keys, all 13 fact IDs present exactly once, all states in `{seen, not_seen, uncertain}`).

fact_accuracy Three-class accuracy over all facts in all samples.

macro_f1 Macro-averaged F1 across three states, per fact, averaged over 13 facts.

seen_f1 F1 for the positive **seen** class, per fact, averaged over 13 facts. This is a key metric because missing or hallucinating a **seen** state directly affects downstream procedure-state reasoning.

sample_exact_match Proportion of samples where all 13 facts match ground truth exactly.

critical_false_positive_count Count of errors where any of 6 critical facts (`fcs_page_x_marks_visible`, `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`, `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`) is predicted **seen** when ground truth is **not_seen** or **uncertain**. These are tracked separately because they represent the most consequential error type: incorrectly reporting a completion-critical state as observed could cause the tutoring system to prematurely advance the procedure.

4.6.2 Base vs. LoRA Comparison

Both configurations receive identical prompts, images, and generation parameters: temperature = 0.0 (greedy decoding), `max_new_tokens` = 1024, seed = 3407. The benchmark runs each sample independently, serialises predictions to JSONL, and computes metrics without post-hoc filtering. A comparison artifact (`comparison.json`) computes delta metrics (LoRA – Base) and an automated recommendation flag.

4.6.3 Benchmark Categories

Each benchmark run is assigned a `benchmark_kind`: `contaminated_dev_set` (evaluation data overlaps with training data pool; used for Run-001 regression only) or `heldout_new_session` (independent capture session with zero training overlap; used for Run-002 newfacts and Run-004 random). This distinction is maintained rigorously throughout the benchmark infrastructure and evaluation chapter.

4.6.4 Benchmark Artifacts

Each benchmark run produces a standardised directory: top-level summary with comparison deltas, per-model metric dictionaries with per-fact scores and confusion matrices, per-sample prediction and error JSONL files, a CSV table of per-fact per-model metrics, a human-readable markdown summary, and automatically generated charts (overall

accuracy, per-fact macro F1, per-fact seen F1, critical false positives, and per-fact confusion matrices for both base and LoRA models). These artifacts constitute the full evidence chain for the technical results reported in Chapter 6.

4.7 Human-Subject Pilot Methodology

A small formative VR pilot study was conducted to gather descriptive evidence on how the tutoring system supports learners during the F/A-18C cold-start procedure compared with unaided practice. The pilot was designed as a preliminary field trial rather than a powered controlled experiment.

4.7.1 Participants and Conditions

Nine participants (P01–P09) were recruited from a convenience sample of DCS users with varied experience levels. Each participant completed one trial in a between-subjects design with two conditions:

With-tutor (n=4). P04, P05, P08, P09. Participants received the full telemetry-grounded tutoring system with overlay highlights, VLM visual fact extraction, and structured help text displayed in the DCS in-game text area. Help was triggered via hotkey.

Without-tutor (n=5). P01, P02, P03, P06, P07. Participants received no tutor assistance. Telemetry was recorded passively for later step inference and error coding, but no overlays or help text were shown.

Assignment to conditions was not randomised; participants were assigned based on scheduling availability. Prior DCS and VR experience varied across participants (see Appendix 9, Table 9.5).

4.7.2 Task and Apparatus

All trials used the F/A-18C cold-start procedure with the full 33-step pack (S01–S33) in DCS World on a Meta Quest 3 via Quest Link. The composite-panel layout (left DDI, AMPCD, right DDI) was rendered on an external monitor for experimenter observation. The tutoring runtime ran on a Windows PC with remote model inference via an OpenAI-compatible endpoint. Each trial ended when the participant either completed

the procedure (confirmed by terminal step completion or tutor declaration) or stopped voluntarily after being unable to progress.

4.7.3 Data Sources and Export Pipeline

For each trial, the experiment export pipeline produced a standardised set of artifacts:

- `trial_summary.csv`: per-trial aggregate metrics (completion status, observed duration, help requests, LLM triggers, VLM calls, overlay executed/rejected, fallback count, step completion counts).
- `step_coding.csv`: per-step coding with auto-coded errors (OM/CO/OR/PA/SV), evidence references, auto-confidence, and human review fields.
- `help_cycles.csv`: per-help-cycle detail (generation mode, vision status, VLM call status, overlay targets, fallback reason) for tutor-condition trials.
- `action_timeline.csv`: timestamped telemetry actions with fused step inference.
- `quality_gate.json`: configuration hash verification, event count, and metadata completeness check.
- `session.json` / `summary.json`: per-trial session metadata and interaction summary.

4.7.4 Error Taxonomy and Coding Policy

All steps were coded under a five-category error taxonomy: **OM** (omission — step not completed), **CO** (commission — step executed incorrectly or out of order), **OR** (order error), **PA** (prompting/assistance error — step completed only after tutor intervention or with incorrect tutor guidance), and **SV** (state violation).

The automated coding pipeline produced initial error labels (`Auto_Error_*` columns), which were then reviewed and corrected by a human coder (`CodexTelemetryReview`) using raw telemetry, help-cycle logs, and action timeline evidence. Human-coded columns (`Error_OM` through `Error_SV`) are the authoritative data source for all reported results.

4.7.5 Data-Quality Notes

Two trials require explicit data-quality documentation:

P08 — auto-summary override. The automated trial summary reported P08 as incomplete (Completed=no, TotalStepsCompleted=5/33). However, manual telemetry review confirmed that all 33 steps reached their required states. The auto-pipeline failed because it required explicit step evidence and gate completion signals that the passive visual inference path could not reconstruct for several vision-priority steps. Following the coding policy, manual coding overrides the auto-summary; P08 is reported as completed (33/33 steps) with zero manual errors. The pre-review coding file (`step_coding.before_codex_log_review.csv`) is preserved for audit in the experiment artifacts.

P02 — questionnaire metadata unresolved. P02’s trial directory contains a questionnaire JSON file with internal participant_id = P01, indicating a file mislabelling during export. P02’s task metrics (steps completed, errors) are derived from `trial_summary.csv` and `step_coding.csv` and are usable. P02 is excluded from any workload, TLX, or confidence summaries because the questionnaire data cannot be confidently attributed.

4.7.6 Analysis Approach

Given the small sample ($n = 9$) and non-random assignment, the analysis is strictly descriptive. No null-hypothesis significance tests are performed. Results are reported as per-participant counts and condition-level summaries. Observed trial durations are reported but cannot be interpreted as completion time comparisons because without-tutor trials were stopped at different points by participants who could not progress further. The analysis focuses on three descriptive questions: (1) whether participants completed the procedure, (2) what categories of procedural error occurred and how they differed between conditions, and (3) how tutor interaction was used (help requests, VLM calls, overlay actions, fallback events).

4.8 Summary

The methodology is characterised by four properties. First, *end-to-end traceability*: every benchmark metric can be traced through training data, human-reviewed labels, and original DCS screenshots. Second, *strict holdout independence*: contaminated development benchmarks and independent holdout benchmarks are explicitly distinguished and interpreted differently. Third, *matched comparison design*: the identical data recipe, benchmark protocol, and evaluation sets are applied across three backbones (Qwen3.5-9B,

Qwen3.6-27B, Gemma-4-31B), enabling controlled comparison of model scale, version, and family effects while holding data and evaluation constant. Fourth, *descriptive pilot analysis*: the human-subject pilot is analysed descriptively with explicit data-quality documentation and no inferential statistical claims.

5 Implementation

This chapter covers implementation-level specifics that directly support the research questions, focusing on the evidence-grounding enforcement mechanism, the deterministic fallback path, the VLM runtime integration, and the replay infrastructure. Implementation details not essential to the core thesis argument—CLI command reference, complete technology stack versions, Lua script inventory, and full hyperparameter configuration—are documented in [Appendix 9](#).

5.1 Technology Stack

The Python codebase uses Python 3.10+ with Poetry for dependency management. Core library dependencies are minimal: `httpx` for HTTP, `PyYAML` for declarative configuration, `jsonschema` (Draft 2020-12) for validation of model outputs and telemetry frames, and `Pillow` for image handling in the VLM pipeline. Testing uses `pytest` with branch coverage.

Model serving is provider-agnostic. The primary path uses an OpenAI-compatible API endpoint (configured for vLLM, llama.cpp, or TGI servers). An Ollama fallback provides local compatibility. A deterministic stub returns fixed responses for testing. Multimodal support is toggled via configuration.

The DCS Lua-side integration comprises scripts for state export (DCS-BIOS protocol), overlay rendering (highlight, clear, and pulse commands via DCS’s cockpit highlight API), and composite-panel screenshot capture. LoRA fine-tuning uses Unsloth with PEFT and TRL; pre-labeling and human review are conducted in Label Studio.

5.2 Evidence Grounding Enforcement

The evidence-grounding mechanism described in [Section 3.3](#) is implemented through three coordinated components.

Dynamic JSON Schema generation. The function `build_help_response_schema` generates the HelpResponse JSON Schema at runtime from the current procedure pack’s step identifiers, UI map targets, and error taxonomy codes. For every overlay target in the schema’s `targets` array, a conditional `if/then` sub-schema requires the `evidence` array to contain at least one item referencing that target. Each evidence item must declare a `type` from the five grounding kinds (`var`, `gate`, `rag`, `delta`, `visual`) and carry a non-empty `ref` matching the corresponding prefix family. The schema is cached keyed on configuration file modification times, avoiding redundant regeneration during long-running sessions.

Response validation and mapping. Model output passes through a narrow JSON extraction pipeline (code fence removal and delimiter-based parsing only; transformations outside this set are rejected), schema validation via `Draft202012Validator`, and allowlist checks on step identifiers and overlay targets. The mapping function validates that every overlay target cited in the response has at least one evidence reference whose prefix matches a key actually present in the help-cycle context. Targets without grounding evidence are rejected, and the rejection is recorded in the response metadata.

Safety constraints. The overlay action executor enforces three rules before dispatching to DCS: only `overlay` actions are executable (no control injection or key emulation); every target is verified against the UI target allowlist; at most one target may be highlighted concurrently. When a new highlight is requested, any prior highlight is automatically cleared.

5.3 Deterministic Fallback

The fallback mechanism operates when no model is available, model output fails JSON extraction or schema validation, or response mapping produces an empty result. It examines the currently active step, evaluates unsatisfied gating rules, identifies missing conditions, and classifies each missing evidence source as a hard blocker (gate explicitly fails) or soft blocker (telemetry source absent). This distinction prevents the tutor from incorrectly claiming a step is unsatisfied when evidence is simply unavailable. The fallback produces a text-only hint restricted to the active step; overlay highlights are suppressed unless a local rule can uniquely determine a valid target from the current telemetry state.

5.4 VLM Visual Fact Extraction Runtime

The VLM runtime operates as a pipeline triggered when the active step is vision-priority (S08, S15, S18). A capture request is dispatched to DCS via UDP; the Lua-side integration renders the three primary cockpit displays into a single composite PNG image. The Python-side buffered vision session maintains a sliding window of recent observations and selects both a pre-trigger frame (most recent frame before the trigger, within a 250 ms sync window) and the trigger frame.

The selected frames are base64-encoded and inserted alongside the fact extraction prompt into a multimodal chat-completion request. The prompt enumerates all 13 facts, describes the three-region composite-panel layout, and provides detailed decision-boundary instructions per fact. The raw model output is parsed, sanitised (unknown keys and invalid fact IDs rejected, states restricted to the three admissible values), and validated against a dynamically generated JSON Schema. Sanitised facts are merged into the runtime vision fact snapshot with sticky preservation and TTL-based expiry.

When the VLM is unavailable or the multimodal request fails, the help cycle proceeds with telemetry and knowledge only—a deliberate degradation property rather than a system failure.

5.5 Replay and Logging Infrastructure

The JSONL event store is the sole persistence mechanism for runtime events, implementing append-only writing with immediate flush after each write for crash-safe logging. The replay module provides two entry points: full simulation using a mock environment adapter that replays pre-recorded observation sequences, and log validation that reconstructs the step state machine and verifies legal state transitions against a procedure pack.

The replay evaluation infrastructure supports suite-driven regression replay: a YAML suite file specifies test cases with expected outcomes, model configuration, and optional vision frame directories. Five replay cases cover distinct scenario types (no-operation, battery-on, engine-off, FCS BIT, INS alignment), each with DCS-BIOS raw captures and per-frame sidecar files. The suite also includes 20 live-fixture regression entries that validate specific evidence-grounding and overlay-target correctness properties (Section 3.6).

Interaction metrics are derived from event logs: stall time per step, help request count,

LLM trigger count, UI click and failure counts, and highlight request and acknowledgment counts.

5.6 Live Runtime Orchestration

The live runtime orchestrates a continuous loop: DCS-BIOS UDP frames are ingested and enriched into normalised state variables; when a help trigger is received (hotkey or UDP message), the procedure engine computes a deterministic step hint, a vision capture is triggered if the active step requires it, BM25 knowledge snippets are retrieved, the full prompt is assembled and sent to the model adapter, the response is validated and mapped to overlay actions, and the complete help cycle is logged. The runtime supports both real-time DCS operation and replay of pre-recorded BIOS captures through a shared code path.

Model access is configured through environment variables with a unified naming convention; the runtime validates all required configuration at startup and redacts API keys from all log output.

6 Evaluation

This chapter presents empirical evidence organised around the three research questions defined in Chapter 1. Section 6.1 (RQ2) reports the VLM technical benchmark results on the 13-fact ontology across three backbone models. Section 6.2 (RQ1) assesses system-level technical readiness through replay evaluation and harness regression testing. Section 6.3 (RQ3) reports the formative VR pilot study with nine participants.

Figure 6.1 gives the chapter-level evidence map. The three evaluation lines use different artifacts and support different kinds of claim: benchmark metrics for RQ2, regression and replay evidence for RQ1, and descriptive pilot data for RQ3.

6.1 RQ2: VLM Technical Evaluation

The technical evaluation addresses whether a LoRA-fine-tuned VLM can reliably extract structured visual facts from F/A-18C cockpit screenshots on holdout data the model has never seen during training. All metrics are computed by the automated benchmark harness described in Section 4.6.

6.1.1 Dataset and Training Summary

The visual fact extraction task evolved across two generations of fact ontology. The first generation (v1) defined 8 visual facts and established the initial training pipeline. The second generation (v2) expanded the ontology to 13 facts, aligned with the runtime contract. Table 6.1 summarises the six annotated datasets.

The v2 training recipe combines Run-003 (220 reviewed tasks) with Run-005 (122 reviewed tasks, collected to rebalance under-represented fact states), with Run-005 applied twice ($\text{Run-005} \times 2$). Bilingual export (EN + ZH) yields a combined training set of **928 rows**. The holdout sets—Run-002 newfacts (50 samples) and Run-004 random (100 samples)—are English-only with verified **zero exact overlap** with training data.

Evaluation data flow across research questions

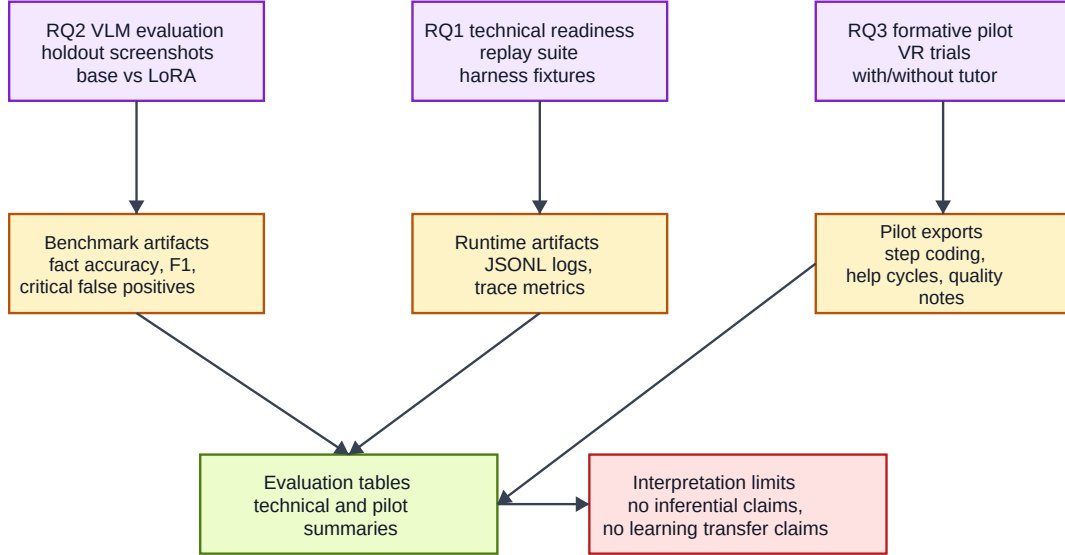


Figure 6.1: Evaluation and data flow across research questions. VLM benchmark artifacts support RQ2, replay and harness logs support RQ1 technical readiness, and VR pilot exports support the descriptive RQ3 findings.

Table 6.1: Summary of annotated visual fact datasets.

Dataset	Run	Facts	Tasks	Bilingual	Role
vision_sft	Run-001	8	180	yes	v1 train
vision_sft_holdout_run002	Run-002	8	50	no	v1 hold
vision_sft_holdout_run002_newfacts	Run-002	13	50	no	v2 hold
vision_sft_run003	Run-003	13	220	yes	v2 training
vision_sft_run005_composition_rebalance	Run-005	13	122	yes	v2 rebal
vision_sft_holdout_run004_random	Run-004	13	100	no	v2 hold

6.1.2 Qwen3.5 13-Fact V2 Results

The primary technical result is the performance of the Qwen3.5-9B LoRA adapter trained on Run-003 + Run-005 $\times$ 2 (928 rows, 4 epochs, learning rate 2×10^{-4} , LoRA $r = 16$, $\alpha = 16$). Table 6.2 reports results on two independent holdout sets.

Table 6.2: 13-fact v2 benchmark: Qwen3.5-9B Base vs. LoRA on two holdout sets.

Metric	Run-002 newfacts (50)			Run-004 random (100)		
	Base	LoRA	Δ	Base	LoRA	Δ
JSON valid	1.0	1.0	—	0.96	1.0	+0.04
Schema valid	1.0	1.0	—	0.96	1.0	+0.04
Fact accuracy	0.8677	0.9908	+0.1231	0.8038	0.9931	+0.1892
Macro F_1	0.4513	0.6515	+0.2002	0.4393	0.6100	+0.1707
Seen F_1	0.5022	0.9648	+0.4626	0.5659	0.9159	+0.3500
Exact match	0.10	0.88	+0.78	0.03	0.92	+0.89
Critical FP	11	4	-7	70	8	-62

The LoRA adapter achieves fact accuracy of 0.9908 (Run-002) and 0.9931 (Run-004), with sample exact match rates of 0.88 and 0.92 respectively. Critical false positives drop by 64% (11 $\rightarrow$ 4) on Run-002 and 89% (70 $\rightarrow$ 8) on Run-004. The base model degrades markedly on the larger Run-004 holdout (4 schema-invalid responses, 70 critical false positives concentrated on `fcsmc_intermediate_result_visible`), while the LoRA adapter restores format reliability and suppresses indiscriminate hallucination.

The residual critical false positives in the LoRA adapter are concentrated on completion-type facts: `fcsmc_final_go_result_visible` (8 on Run-004, 1 on Run-002) and `ins_ok_text_visible` (2 on Run-002).

6.1.3 Gemma-4 and Qwen3.6 Backbone Comparison

To assess whether the training recipe generalises, the identical data recipe (Run-003 + Run-005 $\times$ 2, 928 rows, 13-fact ontology) was applied to two additional backbones. Table 6.3 summarises the comparison.

Qwen3.6-27B’s base model starts substantially stronger than Qwen3.5-9B (fact accuracy 0.91 vs. 0.87 on Run-002) but the LoRA gain is narrower ($\Delta = +0.077$ vs. $+0.123$), consistent with the expectation that larger models have less adaptation headroom under small-data fine-tuning. The Qwen3.6 LoRA adapter achieves comparable performance to Qwen3.5 on most metrics but does not surpass the 9B line.

Table 6.3: 13-fact v2 results across three backbones (LoRA models only).

Metric	Run-002 newfacts (50)			Run-004 random (100)		
	Q3.5-9B	Q3.6-27B	G-4-31B	Q3.5-9B	Q3.6-27B	G-4-31B
Fact accuracy	0.9908	0.9877	0.9492	0.9931	0.9892	0.9715
Seen F_1	0.9648	0.9479	0.8123	0.9159	0.9147	0.7959
Exact match	0.88	0.86	0.44	0.92	0.86	0.74
Critical FP	4	3	0	8	8	13

Gemma-4-31B achieves zero critical false positives on Run-002, reflecting a more conservative error style, but at the cost of lower recall (seen F_1 0.81 vs. 0.96 for Qwen3.5 on Run-002) and lower sample exact match (0.44 vs. 0.88). This backbone-level difference in error style—conservative (Gemma) vs. assertive (Qwen)—is relevant for safety-critical deployment decisions.

Overall, Qwen3.5-9B remains the best-performing configuration on fact accuracy, seen F_1 , and sample exact match across both holdouts. The data recipe transfers across all three backbones with consistent improvements over their respective base models.

6.1.4 Error Analysis

The 13 facts can be grouped into three difficulty categories based on the Qwen LoRA adapter’s performance. **Coarse panel presence** (6 facts: `tac_page_visible`, `supt_page_visible`, `fcs_page_visible`, `bit_root_page_visible`, `fcsmc_page_visible`, `hsi_page_visible`): near-perfect classification, with the single exception of two false negatives on `tac_page_visible` on Run-002. **Fine-grained state indicators** (5 facts: `fcs_page_x_marks_visible`, `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, `fcsmc_final_go_result_visible`, `hsi_map_layer_visible`): perfect on four facts; residual false positives on `fcsmc_final_go_result_visible` (8 on Run-004). **Small-text recognition** (2 facts: `ins_grnd_alignment_text_visible`, `ins_ok_text_visible`): strong on the alignment text (one false positive on Run-002); 2 false positives on `ins_ok_text_visible` on Run-002.

The base model’s dominant failure mode is unidirectional hallucination on rare states: `fcsmc_intermediate_result_visible` appears in only 7 of 100 Run-004 samples, yet the base model asserts it as **seen** in 60 additional samples. The LoRA adapter eliminates this class of error almost entirely, shifting the error profile from diffuse hallucination to focused, interpretable residual errors on completion-type facts.

6.2 RQ1: System-Level Technical Readiness

Beyond the component-level VLM benchmarks, the repository contains evidence of systematic validation at the system level.

6.2.1 Replay Evaluation

The replay evaluation suite at `replay_eval/fa18c_startup_v04/` defines five test cases exercising distinct scenario types: no-operation, battery-on, battery-and-generator-off, FCS reset and BIT, and INS alignment (see Appendix 9, Table 9.4). Each case includes DCS-BIOS raw captures and, where applicable, per-frame sidecar files for vision-dependent steps. All five cases produce the expected step inference and overlay targets, confirming that the telemetry pipeline, procedure engine, and vision path function correctly end-to-end when driven by recorded data.

6.2.2 Harness Fixture Regression

The step harness (Section 3.6) is covered by 20 live-fixture regression entries in the coverage matrix, each replaying a specific telemetry snapshot through the full help cycle and asserting a specific expected outcome. The fixtures cover: correct advancement on completed conditions, safe rejection of wrong-target proposals (e.g., PB18 repaired to PB5 for BIT root sub-state, S10 left-engine guidance using left-click rather than right-click semantics), prevention of VLM evidence leakage into non-visual steps, correct handling of moving/settling controls (refuelling probe, arresting hook), consistent interaction semantics (mouse wheel for rotary controls), and correct repair of partially completed steps.

6.2.3 Experiment Export Infrastructure

The experiment export pipeline (Section 4.7) has been exercised on nine live VR trials, producing complete artifact sets (`trial_summary`, `step_coding`, `help_cycles`, `action_timeline`, `quality_gate`) for each. Quality gates passed for all nine trials, with configuration hashes verified consistent across the study. This demonstrates that the instrumentation pipeline is functional for human-subject data collection, though the participant/session metadata schema remains incomplete (missing questionnaire linkage fields noted in P04's quality gate warnings).

6.3 RQ3: Formative VR Pilot Study

A small formative pilot study was conducted with nine participants to gather descriptive evidence on how the tutoring system supports learners during the F/A-18C cold-start procedure compared with unaided practice. The study should be interpreted as a preliminary field trial, not a powered controlled experiment. Methodology details are reported in Section 4.7.

6.3.1 Participant and Trial Overview

Table 6.4 summarises each participant’s condition, completion status, step completion, observed duration, and key interaction counts. All data are drawn from manual step coding (`step_coding.csv`) and trial summaries (`trial_summary.csv`).

Table 6.4: Participant and trial overview. Observed duration is the wall-clock interval from first telemetry event to last; it is not a completion time measure for incomplete trials.

P	Condition	Completed	Steps	Dur. (s)	Help	VLM	Ovrl.	Fallbk.
P01	without_tutor	no	23/33	2015	0	0	0	0
P02	without_tutor	no	27/33	739	0	0	0	0
P03	without_tutor	no	24/33	243	0	0	0	0
P06	without_tutor	no	29/33	269	0	0	0	0
P07	without_tutor	no	19/33	218	0	0	0	0
P04	with_tutor	yes	33/33	442	19	6	22	6
P05	with_tutor	yes	33/33	317	7	2	6	2
P08	with_tutor	yes*	33/33	332	7	5	10	4
P09	with_tutor	yes	33/33	1033	44	10	44	9

*P08: auto-summary reports incomplete; manual coding confirms 33/33 completed (see data-quality note below).

Completion. All four with-tutor participants completed the full 33-step procedure. None of the five without-tutor participants completed the full procedure; they stopped after completing 19–29 steps. The most commonly missed steps in the without-tutor condition were S18 (FCS-MC BIT page, missed by 5/5 participants), S19 (FCS BIT final GO, missed by 5/5), and S27 (flap switch to AUTO, missed by 5/5).

Observed duration. With-tutor trials ranged from 317 s to 1033 s (median 387 s). Without-tutor trials ranged from 218 s to 2015 s. Because without-tutor trials ended at different procedure points when participants could not progress, these durations cannot be compared as completion times.

6.3.2 Procedural Errors

Table 6.5 reports manual error counts per participant.

Table 6.5: Manual procedural error counts per participant (from manual step coding).
OM = omission, CO = commission, OR = order error, PA = prompting/ assistance error, SV = state violation.

P	Condition	OM	CO	OR	PA	SV
P01	without_tutor	8	0	1	0	1
P02	without_tutor	0	0	0	1	0
P03	without_tutor	7	0	2	0	2
P06	without_tutor	2	0	0	2	0
P07	without_tutor	11	0	0	0	0
P04	with_tutor	0	3	0	3	1
P05	with_tutor	0	3	0	3	0
P08	with_tutor	0	0	0	0	0
P09	with_tutor	0	3	0	2	0

Omission errors. The without-tutor condition shows a clear omission pattern: four of five participants recorded 7–11 OM errors (missed steps). In contrast, the with-tutor condition recorded zero OM errors—all 33 steps were completed by all four participants.

Commission and prompting errors. The with-tutor condition shows a different error profile: CO errors (3 per participant in P04, P05, P09) and PA errors (2–3 per participant) replace the omission errors observed in the without-tutor condition. These are primarily assistance-coupled errors—cases where the tutor provided guidance that was followed but did not fully align with the procedure specification, or where the participant executed a step correctly in substance but with minor deviation from the prescribed sequence.

Error totals. Without-tutor participants averaged 7.4 manual errors; with-tutor participants averaged 4.5. However, the error categories differ qualitatively: without-tutor errors are dominated by omissions (mean OM = 5.6), while with-tutor errors are exclusively CO and PA (mean CO = 2.3, mean PA = 2.0).

6.3.3 Tutor Interaction

Table 6.6 summarises tutor interaction metrics for the four with-tutor trials.

All help cycles used the model generation path (`generation_mode = model`); no cycles fell back to deterministic-only mode. VLM calls were issued for vision-priority

Table 6.6: Tutor interaction summary (with-tutor condition only).

P	Help cycles	VLM calls	Overlay exec.	Overlay rej.	Fallback
P04	19	6	22	0	6
P05	7	2	6	0	2
P08	7	5	10	0	4
P09	44	10	44	0	9
Total	77	23	82	0	21

steps (S08, S15, S18) and totalled 23 across four trials. **Overlay rejection rate was zero:** all 82 overlay actions proposed by the tutor passed evidence-grounding, allowlist, and schema validation. The 21 fallback events ($21/77 = 27\%$ of help cycles) occurred when the model’s proposed overlay targets were repaired by the harness—for example, when the model proposed a target that the harness re-mapped to a more specific sub-state target (e.g., PB18 repaired to PB5 for BIT root).

6.3.4 Data-Quality Notes

1. **P08 auto-summary correction.** The automated pipeline reported P08 as incomplete (5/33 steps). Manual telemetry review confirmed all 33 steps reached required states. The auto-pipeline failed to reconstruct completion evidence for vision-priority steps from passive telemetry. Manual coding overrides the auto-summary; P08 is reported as completed with 0 manual errors.
2. **P02 questionnaire exclusion.** P02’s questionnaire JSON file contains internal `participant_id = P01`, indicating mislabelling. P02 is excluded from any workload, TLX, or confidence summary. Task metrics (steps, errors) from `trial_summary.csv` and `step_coding.csv` are used.
3. **Duration interpretation.** Without-tutor trials ended at different procedure points when participants could not progress further. Observed durations are reported for completeness but do not support completion-time comparison between conditions.
4. **Non-random assignment.** Participants were assigned to conditions based on scheduling availability, not randomisation. Prior DCS and VR experience varied across participants (see Appendix 9, Table 9.5).

6.3.5 Summary of Pilot Findings

The pilot provides three pieces of formative evidence. First, all four tutor-condition participants completed the full 33-step procedure, whereas none of the five without-tutor participants did, suggesting the tutor enables procedural progression for these participants under these conditions. Second, the tutor condition eliminated omission errors in the coded data, but did not eliminate all errors: assistance-coupled CO and PA errors appeared in three of four with-tutor trials. Third, the tutoring infrastructure—telemetry grounding, overlay dispatch, evidence validation—operated without overlay rejections across 77 help cycles, supporting the technical feasibility of evidence-grounded tutoring in a live VR setting.

These findings are descriptive and formative. They do not establish that the tutor improves learning, reduces workload, or accelerates skill acquisition relative to alternative methods. The pilot’s small sample ($n = 9$), non-random assignment, and variable participant experience preclude inferential conclusions. Larger controlled evaluation addressing these limitations is required.

7 Discussion

This chapter interprets the results presented in Chapter 6 and situates them within the larger trajectory of the thesis. Section 7.1 explains why the project evolved from the original VR-first proposal toward the telemetry- and VLM-heavy implementation reported here, and how the formative pilot study fits within this trajectory. Section 7.2 interprets what the benchmark results do and do not demonstrate. Section 7.3 interprets the formative VR pilot findings. Section 7.4 draws design lessons from the two-generation evolution of the visual fact ontology. Section 7.5 enumerates the known limitations of the current system and experimental evidence. Section 7.6 outlines directions for future work that follow from these limitations.

7.1 Evolution from Proposal to Implemented System

The original thesis proposal [20] specified a VR-first grounded tutoring system with gaze, hand-tracking, and voice input on the Meta Quest 3, supported by a lightweight RAG+LLM backend and evaluated through a three-condition human-subject study. The work reported in this thesis differs from that vision in two substantive ways.

First, the implementation effort shifted from VR-native interaction to telemetry-grounded tutoring on desktop DCS, because the core technical challenges—reliable telemetry processing, evidence grounding, VLM-based visual fact extraction, and harness-level output validation—required a stable development and debugging environment that desktop DCS provides. This shift produced the ports-and-adapters architecture, the procedure pack and gating engine, the deterministic fallback path, the JSONL logging and replay infrastructure, and the VLM adaptation pipeline—none of which were anticipated in the original proposal.

Second, the VLM adaptation work grew from an unplanned component into one of the thesis’s main contributions. The original proposal assumed read-only simulator state flags would suffice for procedural state tracking; in practice, several critical procedure steps require visual confirmation of display-page content not exported by DCS-BIOS. Ad-

addressing this gap led to the visual fact ontology, the seven-stage data pipeline, the LoRA fine-tuning experiments, and the three-backbone benchmark comparison—forming a self-contained contribution with verified holdout evidence.

The formative VR pilot study (RQ3) represents a partial convergence back toward the original proposal’s human-subject evaluation ambition. The pilot used the Quest 3 with the full tutoring runtime, produced real participant data, and demonstrated that the evidence-grounded infrastructure operates correctly in a live VR setting. However, the pilot was small ($n = 9$), non-randomised, and descriptive rather than inferential. It does not fulfil the proposal’s three-condition controlled evaluation goal. The gap between the formative pilot and a fully powered controlled study remains the central unfinished thread of the thesis.

7.2 Interpretation of Technical Results

The benchmark results reported in Section 6.1 provide evidence about the performance of the VLM visual fact extraction pipeline. This section draws interpretive conclusions from those results, distinguishing carefully between what the benchmarks demonstrate and what they do not.

7.2.1 What the Benchmark Results Demonstrate

Small-data LoRA adaptation greatly improves domain-specific VLM extraction. Under the 13-fact v2 ontology, the Qwen3.5-9B LoRA adapter trained on 928 rows (Run-003 + Run-005 $\times$ 2) achieves a fact accuracy of 0.9908 on the Run-002 newfacts holdout and 0.9931 on the Run-004 random holdout, compared to 0.8677 and 0.8038 for the base model, respectively. The seen F_1 rises from 0.50–0.57 (base) to 0.92–0.96 (LoRA). These are large effect sizes by any standard, and they are observed on holdout sets with verified zero training overlap, ruling out memorisation as an explanation.

The data recipe transfers across model backbones. The identical training recipe (Run-003 + Run-005 $\times$ 2, bilingual, 13-fact ontology) applied to the Gemma-4-31B backbone produces a similar pattern of consistent improvement: fact accuracy rises from 0.82 to 0.95 on Run-002 and from 0.81 to 0.97 on Run-004. This cross-backbone transfer provides evidence that the improvement is driven by the data and ontology design rather than by idiosyncratic properties of a single model architecture.

JSON and schema validity become near-perfect after training. On the Run-

004 random holdout, the base model produces 4 schema-invalid or JSON-invalid responses out of 100; the LoRA adapter produces none. This is not a trivial property: in a runtime system that must parse model output to extract overlay targets and step recommendations, format reliability is a hard requirement. The fine-tuning process reliably teaches the model to conform to the expected output schema.

The error profile shifts from indiscriminate hallucination to targeted false positives on completion-type facts. The base model’s dominant failure mode is unidirectional hallucination: it asserts `seen` for many facts that are not present, particularly on rare states such as `fcsmc_intermediate_result_visible` (60 false positives on Run-004 out of only 7 true occurrences). The LoRA adapter eliminates this class of error almost entirely, concentrating its residual errors on a specific subset of facts: `fcsmc_final_go_result_visible` (8 false positives on Run-004) and, to a lesser extent, `ins_ok_text_visible` (2 false positives on Run-002). This shift from diffuse hallucination to focused, interpretable error patterns is a qualitative improvement that is not fully captured by aggregate accuracy metrics.

The benchmark protocol itself constitutes a contribution. The use of holdout sets with verified zero overlap, per-fact confusion analysis, critical false positive tracking, and automated chart generation provides a reproducible evaluation framework that can be reused for future iterations of the ontology, for different backbone models, and for other procedures.

7.2.2 What the Benchmark Results Do NOT Demonstrate

They do not demonstrate that the tutoring system improves learning. The benchmarks measure the accuracy of visual fact extraction from cockpit screenshots. Accurate fact extraction is a necessary condition for a grounded tutoring system to function correctly, but it is not a sufficient condition for improving human learning outcomes. A learner could receive perfectly accurate visual facts and still fail to learn the procedure, or could learn effectively from a system with imperfect fact extraction. The causal chain from perception accuracy to learning outcomes has not been tested.

They do not demonstrate that visual fact extraction leads to better tutoring. The benchmarks evaluate the VLM component in isolation, not the end-to-end tutoring system. Whether the availability of visual facts improves the quality, timeliness, or acceptability of the tutor’s help responses relative to a telemetry-only baseline is an open question that requires a system-level evaluation.

They measure a perception subtask, not pedagogical effectiveness. The

metrics reported — fact accuracy, seen F_1 , sample exact match, critical false positives — are perception metrics. They answer the question “does the VLM see the cockpit correctly?” rather than “does the tutoring system help the learner?”. Conflating the two would be a category error.

7.2.3 The `ins_go` to `ins_ok_text` Decomposition as a Design Lesson

The 8-fact v1 ontology included a single fact, `ins_go`, which asked the VLM to recognise whether the INS alignment had reached a visible GO state. This fact proved to be the dominant source of false positives in the v1 adapter: 13 of 50 Run-002 samples were incorrectly predicted as `seen`. The issue was not that the VLM failed to learn from the training data, but that the target itself was poorly specified. `ins_go` was not a visual fact in the same sense as “the FCS page is visible on the left DDI”; it was a procedural completion judgement that required integrating the presence of specific text (“OK”), the absence of other text (alignment countdown), and the procedural context (which phase of the alignment the system is in). Asking a single-frame VLM to make this judgement conflated perception with reasoning.

The 13-fact v2 ontology decomposes this compound target into lower-level, visually observable facts: `ins_grnd_alignment_text_visible` and `ins_ok_text_visible` each describe a specific piece of text on the AMPCD. The procedural interpretation — whether the combination of these facts indicates that alignment is complete — is deferred to the downstream procedure engine, which has access to telemetry, procedure phase, and timing information. This decomposition was not a cosmetic change; it was the single most important design decision in the ontology evolution, and the benchmark results bear out its effectiveness: `ins_grnd_alignment_text_visible` achieves a seen F_1 of 0.99 on Run-002, and `ins_ok_text_visible` produces only 2 false positives on Run-002 and zero on Run-004.

7.3 Interpretation of Pilot Findings

The formative VR pilot study (Section 6.3) provides descriptive evidence that must be interpreted within its proper scope.

Completion and omission errors. The clearest signal is the completion gap: 4/4 with-tutor participants completed all 33 steps, while 0/5 without-tutor participants did.

Coupled with the error coding—zero OM errors in the tutor condition versus a mean of 5.6 OM errors in the without-tutor condition—this suggests the tutor helped participants progress through the procedure by reducing missed steps. This is consistent with the tutor’s design: the help system actively identifies the next required step and highlights the relevant cockpit control, making it harder for a participant to skip a step unknowingly.

Assistance-coupled errors. The tutor condition did not eliminate errors; it changed their composition. CO and PA errors appeared in three of four with-tutor trials, with a total of 9 CO and 8 PA errors across the condition. These are primarily assistance-coupled: cases where the tutor provided guidance that was followed but where the resulting action did not fully align with the procedure specification. For example, the harness occasionally repaired model-proposed overlay targets (PB18 to PB5 for BIT root sub-state), and these repairs, while correct, could produce a participant action that was technically the right control but in a slightly different context than the strict procedure definition. These errors highlight the inherent tension between flexible LLM-generated guidance and fixed procedural specifications: the tutor can guide correctly in substance while deviating in form.

Infrastructure reliability. The zero overlay rejection rate (0/82) across 77 help cycles provides evidence that the evidence-grounding, allowlist, and schema validation pipeline functions correctly in live VR operation. All 21 fallback events were harness repairs (target re-mapping), not rejections. This supports the architectural claim that evidence grounding can be enforced at the schema level without blocking legitimate tutor output.

What the pilot does not show. The pilot provides no evidence about learning (no pre/post knowledge measures), retention (no follow-up testing), workload (P02 questionnaire data unresolved; other questionnaire data not yet fully coded), or transfer. The small sample, non-random assignment, and variable participant experience mean that observed differences between conditions cannot be attributed to the tutor alone. The pilot’s value is formative: it demonstrates that the instrumentation pipeline captures the intended data, that the tutor functions end-to-end in VR, and that the experimental design is executable.

7.4 Ontology Design Lessons

The two-generation evolution from 8-fact v1 to 13-fact v2 provides concrete design lessons that may inform future visual ontology construction for procedural training systems.

7.4.1 Abstraction-Level Discipline

The central lesson is that a VLM-based visual fact extractor should be asked to report *observable visual evidence*, not *procedural conclusions*. The 8-fact v1 ontology mixed these two levels of abstraction within a single fact set: facts such as `fcs_page_visible` and `bit_root_page_visible` described visually localisable display-page states, while facts such as `ins_go` and `fcs_bit_result_visible` encoded higher-level procedural judgements. The mixed-abstraction ontology produced the predictable consequence: straightforward page-visibility facts achieved strong performance, while the compound procedural facts accounted for the majority of critical false positives.

The 13-fact v2 ontology enforces a design discipline in which every fact is a locally verifiable piece of visual evidence. This discipline has three practical consequences. First, it makes the per-fact annotation task clearer for human reviewers: deciding whether “OK” text is visible on the AMPCD is a well-defined visual judgement; deciding whether “INS alignment is complete” is not. Second, it produces cleaner training signals, because the mapping from image content to fact label is more direct and less dependent on implicit procedural context. Third, it localises errors: when a fact is misclassified, the source of the error can be inspected in the image region that the fact describes, without needing to reason about the model’s implicit understanding of procedure phase.

General design principle. The experience of moving from a mixed-abstraction ontology (v1) to a disciplined one (v2) yields a design principle that is explicit and transferable:

Design Principle: VLM fact targets should be low-level, locally verifiable visual propositions. Higher-level procedural state judgments should be synthesised by downstream reasoning from multiple visual facts combined with temporal and telemetry context.

This principle formalises the boundary between perception and reasoning. The VLM is responsible for answering “what text, symbol, or display element is visible in this image region?”—questions that have ground-truth answers determinable from a single frame. The procedure engine is responsible for answering “what does this combination of visual

facts, telemetry variables, and temporal context imply about the current procedure state?”—questions that require multi-source integration. Enforcing this boundary at the ontology design stage, rather than discovering it through error analysis after training, is the primary design lesson of the ontology evolution.

7.4.2 Decomposition of Compound Targets

The effectiveness of this decomposition is measurable. On the Run-002 holdout, the 8-fact v1 LoRA adapter produced 15 critical false positives, of which 13 (87%) were on the compound target `ins_go`. After decomposing `ins_go` into the low-level visual propositions `ins_grnd_alignment_text_visible` and `ins_ok_text_visible`, the 13-fact v2 LoRA adapter reduced the critical false positive count from 15 to 4, with the combined residual errors on these two decomposed facts falling to 3 (1 on `ins_grnd_alignment_text_visible`, 2 on `ins_ok_text_visible`). This is not simply a change of labels; it is a measurable improvement—from 13 critical false positives to 3 on the same underlying visual content—that demonstrates the effect of applying the design principle in practice.

The `ins_go` $\rightarrow$ `{ins_ok_text_visible, ins_grnd_alignment_text_visible, hsi_map_layer_vis}` decomposition is not the only example of this principle. The v1 fact `fcs_bit_result_visible` was similarly a compound target that asked the VLM to recognise a final test result rather than the visual indicators that characterise it. In v2, this is decomposed into `fcsmc_intermediate_result_visible`, `fcsmc_in_test_visible`, and `fcsmc_final_go_result_vis`, each capturing a visually distinguishable stage in the FCS-MC test sequence. The downstream procedure engine can then interpret the sequence of these facts — together with telemetry state — to determine whether the FCS BIT is complete.

The general design principle can be stated as follows: if a human annotator cannot determine the fact label by looking at a single image without knowing the procedure step, the fact is likely too high-level and should be decomposed. This principle is operationalisable and transferable to other procedures. While demonstrated on cockpit displays, this principle applies to any VLM-based perception system that must distinguish visual presence from semantic state—for example, medical imaging (tissue visible vs. condition diagnosed), industrial inspection (gauge visible vs. value in range), and document understanding (signature field visible vs. form completed).

7.4.3 Residual Failure Modes and Their Implications

Despite the ontology redesign, completion-type facts remain the most challenging category. On the Run-004 random holdout, all 8 residual critical false positives for the Qwen LoRA adapter are on `fcsmc_final_go_result_visible`. This fact requires the model to recognise a specific numeric readout (the final GO result) and to distinguish it from visually similar intermediate numeric readouts that appear in the same screen region.

There are two plausible interpretations of this residual difficulty. One is that it is a data coverage problem: the number of training examples in which the final GO result is visible is small, and the visual distinction between intermediate and final readouts may not be sufficiently represented. The other is that it is a fundamental resolution limitation: at the image resolution used for training and inference, the difference between an intermediate numeric value and the final GO value may be genuinely below the discriminative threshold of the model. Disambiguating these two interpretations would require a controlled experiment that varies image resolution and training-set composition independently, which is beyond the scope of the current work.

7.4.4 Implications for Future Ontology Design

The experience of designing two generations of visual fact ontology for the F/A-18C cold-start suggests three guidelines for ontology construction in related procedural domains:

1. **Begin with a task analysis** that identifies which procedure states require visual confirmation and which can be covered by telemetry or rule-based inference. The ontology should only contain facts for states that genuinely require visual evidence.
2. **Enforce a single-frame verifiability test** during ontology design: every fact should be evaluable from a single cockpit image without procedural context. Facts that fail this test should be decomposed or deferred to downstream reasoning.
3. **Collect composition-rebalanced data** early in the process, particularly hard negatives that expose the boundary between visually similar but procedurally distinct states. The Run-005 composition rebalance was added specifically to address under-represented fact states, and its inclusion in the training recipe was critical to the reduction in hallucination.

7.5 Limitations

The following limitations bound the scope of conclusions that can be drawn from the present work. They are stated explicitly so that the contribution can be evaluated within its proper scope and so that future work can address them systematically.

7.5.1 Limitations of the Experimental Setup

Single procedure. All experiments, benchmarks, and dataset curation efforts focus on a single procedure: the F/A-18C cold-start. There is no evidence that the 13-fact ontology, the training recipe, or the LoRA adaptation approach generalises to other procedures (e.g., taxi, takeoff, landing, or emergency procedures in DCS, or procedural tasks in other simulators). The ontology design principles articulated in Section 7.4 are plausible but untested beyond the cold-start domain.

Single composite-panel layout. The visual fact extractor is trained and evaluated on a fixed three-display composite panel (left DDI, AMPCD, right DDI). Cockpit configurations that differ in the number, arrangement, or type of displays (e.g., the F-16C with two MFDs, or the A-10C with a different instrument layout) would require retraining with new data under a new layout description. The current models have not been tested on such configurations.

Small dataset sizes. The largest training configuration uses 928 rows (Run-003 + Run-005 $\times$ 2, bilingual). This is small by comparison with typical VLM fine-tuning datasets, which often use tens or hundreds of thousands of examples (e.g., LLaVA instruction tuning and broader instruction-tuned VLM recipes) [5, 14]. While the results demonstrate that 928 rows can produce meaningful improvements, they do not establish the upper bound of what is achievable with more data. Conversely, the results do not establish the minimum data requirement; it is possible that comparable performance could be achieved with fewer examples, which would be practically relevant for domains where annotated data is scarce.

Holdout sets from the same aircraft and cockpit generation. The holdout sets (Run-002 and Run-004) are independent sessions with verified zero overlap with the training data, but they are drawn from the same aircraft type, the same cockpit configuration, and the same rendering engine (DCS World). The benchmarks therefore measure *within-domain* generalisation (new sessions for the same cockpit), not *cross-domain* generalisation (new cockpit types or new simulators).

7.5.2 Limitations of the System

VR runtime operational; pilot completed but controlled study pending. The VR runtime has been exercised in a formative pilot study with nine participants (Section 6.3), confirming that the tutoring system functions end-to-end in live VR operation. However, the pilot was small ($n = 9$), non-randomised, and descriptive. In-headset overlay rendering and latency characteristics under study conditions have been observed but not systematically measured.

No controlled human-subject evaluation. The formative pilot provides descriptive evidence of feasibility but does not constitute a controlled evaluation. The pilot provides no evidence about whether the tutoring system improves learning, retention, transfer, or workload relative to baseline conditions. The causal chain from tutoring system use to improved human learning outcomes remains untested. A larger controlled study with randomised assignment, pre-registered analysis, and adequate statistical power is required to establish pedagogical value.

The benchmark measures fact extraction, not tutoring quality. The VLM benchmarks reported in Section 6.1 are perception metrics computed on isolated single-frame fact extraction. They do not measure the quality, usefulness, or timeliness of the tutor’s help responses, the correctness of overlay targets, or the coherence of multi-turn help sequences. The pilot study addresses some of these gaps at a formative level, but an end-to-end evaluation of the full help cycle under controlled conditions has not been conducted.

7.5.3 Limitations of the Pilot Study

Small sample and non-random assignment. The pilot’s nine participants were assigned to conditions based on scheduling availability, not randomisation. With four with-tutor and five without-tutor participants, condition sizes are small and unbalanced. No inferential statistical tests are appropriate.

Variable participant experience. Participants ranged from novice to frequent DCS users. Prior experience was not balanced across conditions, and the small sample precludes statistical control for experience effects.

Incomplete workload and questionnaire data. P02’s questionnaire file was mislabelled and could not be confidently attributed; workload, TLX, and confidence data are therefore incomplete. NASA-TLX integration was not yet implemented in the export pipeline for these trials.

Without-tutor duration not comparable. Without-tutor trials ended at different procedure points (19–29 steps completed), so observed durations do not measure completion time and cannot be compared with with-tutor durations.

Passive visual verification limitation. For without-tutor trials, step completion was inferred from passive telemetry gates and action timeline review, without active VLM visual confirmation. Steps requiring visual evidence (S08, S18) may be misclassified as completed or not completed based on incomplete telemetry evidence alone.

7.5.4 Limitations of the VLM Adaptation Results

Residual critical false positives on completion-type facts. As discussed in Section 7.4, the Qwen LoRA adapter retains critical false positives on `fcsmc_final_go_result_visible` (1 on Run-002, 8 on Run-004). In a runtime deployment, each such false positive carries the risk that the tutoring system incorrectly concludes that the FCS-MC test is complete and advances the procedure prematurely. Mitigation strategies — such as the sticky-fact TTL mechanism and the requirement for multiple confirming observations — exist but have not been evaluated end-to-end.

Limited model diversity. Two backbone models were evaluated (Qwen3.5-9B and Gemma-4-31B), both with LoRA fine-tuning using the Unsloth + PEFT + TRL stack. The results do not generalise to other model architectures (e.g., LLaVA-style models [14]), other fine-tuning methods (e.g., full fine-tuning, QLoRA with different rank configurations), or other training frameworks.

No ablation of training components. The training recipe combines several design choices — bilingual SFT, Run-005×2 oversampling, composition rebalancing, LoRA rank and alpha settings — without isolating the contribution of each. It is therefore unclear which components are necessary and which are incidental. A systematic ablation study would clarify the design space but has not been conducted.

7.5.5 Scope Summary

These limitations bound the claims of this thesis. Within these bounds, the thesis demonstrates: (1) a working dual-platform tutoring runtime with automated tests and harness-level evidence validation; (2) a reproducible VLM adaptation pipeline achieving 0.99 fact accuracy on holdout data with cross-backbone evidence; (3) a pack-driven architecture supporting extension to new procedures and simulators; and (4) formative VR pilot evidence ($n = 9$) that the tutor was associated with full procedural comple-

tion and fewer coded omission errors, while assistance-coupled errors persisted. These contributions do not depend on claims that exceed the stated bounds.

7.6 Future Work

The limitations enumerated above define a concrete research agenda. This section outlines the principal directions for future work, ordered from the most immediate technical extensions to the longer-term thesis objectives.

7.6.1 Controlled Human-Subject Evaluation

The formative pilot study (Section 6.3) demonstrated feasibility and instrumentation readiness but did not provide controlled evidence about learning, workload, or retention. The next research stage is a larger controlled between-subjects evaluation comparing the grounded tutor against an appropriate baseline condition (e.g., without-tutor or static checklist). This requires completing the participant/session metadata schema, NASA-TLX integration, quiz linkage, and anonymisation pipeline; recruiting an adequate sample with randomised assignment; and pre-registering the analysis plan. Given the pilot’s observed effect on completion rates, a two-condition design (with-tutor vs. without-tutor) with $n \geq 15$ per condition would provide a more feasible starting point than the original three-condition proposal design.

7.6.2 Extending the Visual Fact Ontology to Other Procedures

The current 13-fact ontology is specific to the F/A-18C cold-start. Extending the approach to other procedures — within DCS (taxi, takeoff, landing, aerial refuelling, emergency procedures) or in other simulators — would test the generality of the ontology design principles proposed in Section 7.4. Each new procedure would require a task analysis to identify vision-priority steps, a new fact ontology designed under the single-frame verifiability constraint, capture and annotation of new training data, and independent holdout evaluation. Whether the data recipe (bilingual SFT, composition rebalancing, moderate oversampling) transfers to procedurally different domains is an open empirical question.

7.6.3 Multi-Frame Temporal Reasoning

The current visual fact extraction operates on single composite-panel images. Several cockpit states of interest involve temporal dynamics that cannot be disambiguated from a single frame: countdown values on the INS alignment page, animation sequences on the FCS-MC test display, and transient warning indicators. Extending the VLM input to short frame sequences (e.g., 3–5 consecutive frames at 1–2-second intervals) could enable the model to capture these temporal cues directly. This extension would require changes to the data capture pipeline (recording frame sequences rather than isolated screenshots), the training format (video or multi-image inputs), and the benchmark protocol (temporal fact definitions). Whether current small-scale VLMs can effectively exploit temporal information for cockpit state recognition is uncertain. Whether multi-frame VLM input is feasible at the 9B-parameter scale under LoRA training has not been tested.

7.6.4 Online Adaptation from In-Situ Corrections

The current VLM adaptation pipeline is an offline process: data is captured, pre-labeled, reviewed, and used to train a LoRA adapter in a batch cycle. In a deployed tutoring system, a learner or instructor may correct the system’s visual fact predictions in real time (e.g., “the OK text is not actually visible yet”). Collecting these in-situ corrections and using them for online or incremental model adaptation — without requiring a full re-labeling and re-training cycle — could enable the system to improve its extraction accuracy over the course of a single training session or across multiple learners. This direction requires addressing challenges in label reliability (learner corrections may themselves be erroneous), catastrophic forgetting (online updates should not degrade performance on previously correct facts), and evaluation (online adaptation must be benchmarked against a fixed holdout set that remains untouched by the adaptation process). Whether the current Unsloth + PEFT + TRL stack supports incremental fine-tuning without full retraining has not been evaluated.

7.6.5 System-Level End-to-End Evaluation

Beyond the component-level VLM benchmarks reported in this thesis, a system-level evaluation of the full help cycle is needed. This would measure, under replay or live conditions: the end-to-end latency from help trigger to overlay rendering; the correctness of overlay targets on vision-priority steps; the rate of deterministic fallback (when model

output fails validation); and the perceived usefulness of help responses as judged by domain experts reviewing replay logs. Such an evaluation would bridge the gap between the component-level benchmarks (reported here) and the pilot study (Section 6.3), providing evidence about whether the system’s components function correctly when integrated.

7.6.6 Summary of Future Directions

Table 7.1 summarises the principal future directions and their current status.

Table 7.1: Summary of future work directions.

Direction	Current status	Key challenge
Controlled human-subject evaluation	Formative pilot completed ($n = 9$); controlled study pending	Recruitment, randomisation, statistical power
Ontology extension to other procedures	13-fact v2 validated for cold-start only	Task analysis, annotation effort, domain generality
Multi-frame temporal reasoning	Single-frame pipeline operational	VLM capacity, training data volume, benchmark design
Online adaptation from corrections	Not implemented	Label reliability, catastrophic forgetting, evaluation
System-level end-to-end evaluation	Not conducted	Defining meaningful system-level metrics

The work presented in this thesis provides the technical foundation for each of these directions. The architecture, data pipeline, training infrastructure, and benchmark protocol are designed to be extensible, and the lessons learned from the two-generation ontology evolution and the cross-backbone comparison provide evidence-based guidance for the next stages of the project.

8 Conclusion

This thesis set out to design, implement, and technically validate a telemetry-grounded tutoring runtime for procedural learning in a high-fidelity flight simulator, and to gather formative evidence about its use in immersive VR training. The three contributions delivered are: an evidence-grounded tutoring architecture, a visual fact ontology and small-data VLM adaptation pipeline with cross-backbone benchmark evidence, and a VR evaluation framework exercised in a formative pilot study ($n = 9$). This chapter summarises the completed work against the three research questions, restates the contributions, and outlines directions for future work.

8.1 Summary of Completed Work

The thesis has produced three contributions, corresponding to the three research questions defined in Chapter 1.

C1 — An evidence-grounded procedural tutoring architecture. The tutoring runtime enforces that every overlay target and step recommendation carries a traceable evidence reference (telemetry variable, gating rule, RAG snippet, or visual fact), enforced at the schema level through runtime-generated JSON Schema with **if/then** rules—not merely logged for post-hoc audit. The architecture follows a ports-and-adapters pattern with a deterministic fallback path, a step harness that validates and repairs model outputs before they reach the simulator, and a procedure pack format that expresses all procedure logic as declarative YAML configuration. The architecture is supported by automated tests, a replay evaluation suite (5 cases), and 20 live-fixture regression entries covering correct advancement, safe rejection, and interaction semantics. In the VR pilot, the architecture processed 77 help cycles with zero overlay rejections.

C2 — A visual fact ontology and small-data VLM adaptation pipeline. The 13-fact v2 ontology decomposes cockpit display-page state into locally verifiable propositions. The ontology evolved from an 8-fact v1 baseline that mixed abstraction levels, producing systematic false positives on completion-type facts. The v2 decomposition—

breaking `ins_go` into `ins_grnd_alignment_text_visible` and `ins_ok_text_visible`, and decomposing the FCS-MC sequence into four sub-states—reduced critical false positives by 64% (13→4) on Run-002 and 89% (70→8) on Run-004. A Qwen3.5-9B LoRA adapter trained on 928 bilingual rows achieves 0.99 fact accuracy on both holdout sets. The identical data recipe transfers across three backbones (Qwen3.5-9B, Qwen3.6-27B, Gemma-4-31B) with consistent improvements over their respective base models. The ontology evolution yields a transferable design principle: VLMs should extract observable visual evidence, while procedural interpretation belongs in downstream reasoning.

C3 — A VR evaluation framework supported by a formative pilot. The thesis delivers study-ready instrumentation for evaluating grounded procedural tutoring in immersive simulation, and a formative VR pilot study ($n = 9$) that exercised this instrumentation. All four tutor-condition participants completed the full 33-step procedure; none of the five without-tutor participants did. With-tutor errors were predominantly assistance-coupled (CO/PA), while without-tutor errors were predominantly omissions. The pilot provides descriptive evidence of feasibility and instrumentation readiness, not inferential validation of learning effects. Larger controlled evaluation remains future work.

What this thesis demonstrates. The work establishes that a telemetry-grounded, evidence-constrained tutoring runtime with structured visual fact extraction is technically feasible and can be assessed through systematic benchmarks. It demonstrates that 928 rows of human-reviewed data suffice to adapt a general-purpose VLM to a domain-specific extraction task with near-ceiling accuracy on independent holdout sets. It shows that visual fact ontology design—specifically, the decomposition of compound procedural targets into locally verifiable atoms—has a measurable, quantitative impact on model reliability. The cross-backbone results confirm that the data recipe, not a specific architecture, drives the gains. The formative VR pilot provides initial descriptive evidence that full procedural completion was observed in all tutor-condition trials while omission errors were less frequent, and supports the readiness of the instrumentation pipeline for live VR data collection.

Scope and boundaries. The VLM benchmarks measure fact extraction accuracy, not pedagogical effectiveness. The VR pilot provides formative descriptive evidence ($n = 9$, non-randomised, unbalanced conditions) and does not establish learning gains, workload reduction, or retention improvement. Without-tutor observed durations are not compar-

able with with-tutor durations because trials were stopped at different procedure points. Questionnaire and workload data remain incomplete (P02 questionnaire mislabelled). Within these boundaries, the thesis delivers a technically evaluated tutoring runtime, a reproducible VLM adaptation pipeline with benchmark evidence, and formative VR pilot results that demonstrate feasibility and instrumentation readiness.

8.2 Future Work

The contributions of this thesis open several directions for future work, each of which builds directly on the completed contributions.

Controlled human-subject evaluation. The formative VR pilot study (Section 6.3) demonstrated feasibility and instrumentation readiness. The next step is a larger controlled between-subjects evaluation comparing the grounded tutor against an appropriate baseline, with randomised assignment, pre-registered analysis, and adequate sample size. Key infrastructure upgrades required include completion of the participant/session metadata schema, NASA-TLX integration, quiz linkage, and anonymisation pipeline. Given the pilot’s observed completion-rate pattern, a two-condition design (with-tutor vs. without-tutor) with $n \geq 15$ per condition provides a feasible starting point.

Ontology extension to other procedures. The 13-fact v2 ontology is specific to the F/A-18C cold-start task and its three-display composite panel layout. Extending the ontology to other procedures within DCS—such as navigation, weapon employment, or emergency checklists—would test the generality of the visual fact decomposition strategy and require new dataset capture and annotation cycles. The procedure pack format is designed to accommodate such extensions: a new pack directory with its own `vision_facts.yaml`, `vision_layout.yaml`, `step_registry.yaml`, and `telemetry_map.yaml` would define a new training scenario without changes to the pipeline tools.

Multi-frame temporal reasoning. The current VLM pipeline operates on single frame pairs (pre-trigger and trigger frames) and makes independent predictions per observation window. Incorporating temporal context across multiple frames could improve accuracy on facts that involve transient states or state transitions, such as the FCS-MC test sequence (`fcsmc_in_test_visible` $\rightarrow$ `fcsmc_final_go_result_visible`). Pos-

sible approaches include frame stacking as multi-image input to the VLM, recurrent aggregation of fact predictions over a sliding window, or a lightweight temporal model that consumes per-frame fact vectors and produces temporally consistent sequences.

Online adaptation. The current LoRA adapter is trained offline on a fixed dataset and deployed as a static model. An online adaptation loop, in which the system collects new labelled frames during live tutoring sessions (with tutor or expert verification) and periodically updates the adapter, could progressively improve accuracy on edge cases and adapt to changes in cockpit display configurations across DCS versions or aircraft variants. This would require lightweight fine-tuning infrastructure that can operate concurrently with the tutoring runtime and a quality-control mechanism for newly acquired labels.

Deterministic procedure engine enhancements. The current gating rule engine uses a v1 DSL with four operators (`var_gte`, `arg_in_range`, `flag_true`, `time_since`). Extending the DSL with additional operators (e.g., logical combinations, sequence counters, multi-variable conditions) and supporting user-defined gating functions would increase the expressiveness of procedure packs and reduce reliance on manual-step classification for edge cases.

The work completed in this thesis establishes that a telemetry-grounded, evidence-constrained tutoring runtime with structured visual fact extraction is technically feasible and can be assessed through automated benchmarks. The next phase of research must determine whether this capability translates into improved human learning in immersive simulation environments.

References

- [1] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as i can, not as i say: Grounding language in robotic affordances. In *Proceedings of the 6th Conference on Robot Learning*, 2022. doi: 10.48550/arXiv.2204.01691.
- [2] Vincent Aleven, Bruce M. McLaren, Jonathan Sewall, and Kenneth R. Koedinger. A new paradigm for intelligent tutoring systems: Example-tracing tutors. *International Journal of Artificial Intelligence in Education*, 19(2):105–154, 2009. doi: 10.3233/IRG-2009-19(2)02.
- [3] Saleema Amershi, Dan Weld, Mihaela Vorvoreanu, Adam Fourney, Besmira Nushi, Penny Collisson, Jina Suh, Shamsi Iqbal, Paul N. Bennett, Kori Inkpen, Jaime Teevan, Ruth Kikin-Gil, and Eric Horvitz. Guidelines for human-AI interaction. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems*, pages 1–13, 2019. doi: 10.1145/3290605.3300233.
- [4] Jinze Bai, Shuai Bai, Shusheng Yang, Shijie Wang, Sinan Tan, Peng Wang, Junyang Lin, Chang Zhou, and Jingren Zhou. Qwen-VL: A versatile vision-language model for understanding, localization, text reading, and beyond. *arXiv preprint arXiv:2308.12966*, 2023.
- [5] Wenliang Dai, Junnan Li, Dongxu Li, Anthony Meng Huat Tiong, Junqi Zhao, Weisheng Wang, Boyang Li, Pascale Fung, and Steven Hoi. InstructBLIP: Towards general-purpose vision-language models with instruction tuning. In *Advances in Neural Information Processing Systems*, volume 36, 2023. doi: 10.48550/arXiv.2305.06500.
- [6] DCS-BIOS Project. DCS-BIOS documentation, n.d. DCS-BIOS v0.10.0 documentation. Accessed 2026-06-05. Available at: <https://dcs-bios.readthedocs.io/en/latest/>.

-
- [7] Eagle Dynamics. DCS World, 2013. Digital Combat Simulator. Available at: <https://www.digitalcombatsimulator.com/>.
 - [8] Eagle Dynamics. DCS: F/A-18C early access guide, 2023. Updated 18 July 2023. Available at: https://www.digitalcombatsimulator.com/en/downloads/documentation/dcs-hornet_early_access_guide_en/.
 - [9] David Gunning, Mark Stefik, Jaesik Choi, Timothy Miller, Simone Stumpf, and Guang-Zhong Yang. XAI—explainable artificial intelligence. *Science Robotics*, 4(37):eaay7120, 2019. doi: 10.1126/scirobotics.aay7120.
 - [10] Robert T. Hays, John W. Jacobs, Carolyn Prince, and Eduardo Salas. Flight simulator training effectiveness: A meta-analysis. *Military Psychology*, 4(2):63–74, 1992. doi: 10.1207/s15327876mp0402_1.
 - [11] Edward J. Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. LoRA: Low-rank adaptation of large language models. In *International Conference on Learning Representations (ICLR)*, 2022.
 - [12] Enkelejda Kasneci, Kathrin Seßler, Stefan Küchemann, Maria Bannert, Daryna Dementieva, Frank Fischer, Urs Gasser, Georg Groh, Stephan Günnemann, Eyke Hüllermeier, Stephan Krusche, Gitta Kutyniok, Tilman Michaeli, Claudia Nerdel, Jürgen Pfeffer, Oleksandra Poquet, Michael Sailer, Albrecht Schmidt, Tina Seidel, Matthias Stadler, and Gjergji Kasneci. ChatGPT for good? on opportunities and challenges of large language models for education. *Learning and Individual Differences*, 103:102274, 2023. doi: 10.1016/j.lindif.2023.102274.
 - [13] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.
 - [14] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *Advances in Neural Information Processing Systems*, 36, 2023. doi: 10.48550/arXiv.2304.08485.
 - [15] Sourab Mangrulkar, Sylvain Gugger, Lysandre Debut, Younes Belkada, Sayak Paul, and Benjamin Bossan. PEFT: State-of-the-art parameter-efficient fine-tuning methods, 2022. URL <https://github.com/huggingface/peft>.

-
- [16] Antonija Mitrovic. Fifteen years of constraint-based tutors: What we have achieved and where we are going. *User Modeling and User-Adapted Interaction*, 22(1–2):39–72, 2012. doi: 10.1007/s11257-011-9105-9.
 - [17] Riccardo Palmarini, John Ahmet Erkoyuncu, Rajkumar Roy, and Hosein Torabmostaedi. A systematic review of augmented reality applications in maintenance. *Robotics and Computer-Integrated Manufacturing*, 49:215–228, 2018. doi: 10.1016/j.rcim.2017.06.002.
 - [18] Kurt VanLehn. The relative effectiveness of human tutoring, intelligent tutoring systems, and other tutoring systems. *Educational Psychologist*, 46(4):197–221, 2011. doi: 10.1080/00461520.2011.611369.
 - [19] Bryan Wang, Gang Li, Xin Zhou, Zhourong Chen, Tovi Grossman, and Yang Li. Screen2Words: Automatic mobile UI summarization with multimodal learning. *arXiv preprint arXiv:2108.03353*, 2021. doi: 10.48550/arXiv.2108.03353.
 - [20] Yu Zhang. Explain reality: Grounded procedural tutoring with foundation models and visual fact extraction in immersive simulation — MSc thesis proposal. TU Clausthal, Institute for Software and Systems Engineering (ISSE). Unpublished manuscript., 2025.

9 Appendix

9.1 Full LoRA Hyperparameter Configuration

Table 9.1 lists the complete hyperparameter configuration for all three backbone models under the matched Run-003 + Run-005×2 recipe. Values are drawn from available training reports and benchmark artifacts.

Table 9.1: Complete LoRA fine-tuning hyperparameters across three backbones.

Parameter	Qwen3.5-9B	Qwen3.6-27B	Gemma-4-31B
max_seq_length	4096	4096	4096
num_train_epochs	4.0	4.0	4.0
learning_rate	2×10^{-4}	2×10^{-4}	2×10^{-4}
per_device_train_batch_size	1	1	1
gradient_accumulation_steps	4	4	4
lora_r	16	16	16
lora_alpha	16	16	16
lora_dropout	0.0	0.0	0.0
seed	3407	3407	3407
load_in_4bit	True	True	True
warmup_ratio	0.1	0.1	0.1
weight_decay	0.01	0.01	0.01
train_rows	928	928	928
eval_rows	0	0	0
Backbone-specific			
Vision LoRA	enabled	enabled	disabled
LoRA target modules	vision+lang	vision+lang	all-linear
Chat template	Qwen default	Qwen default	gemma-4
GPU memory util.	0.6	0.6	0.95
Training outcomes			
Train runtime (s)	6419	12619	8050
Train loss (final)	0.2156	0.1068	0.0474

Note: training loss values are not directly comparable across backbones due to differ-

ences in tokenizer behaviour, chat-template structure, target-sequence distribution, and model parameterisation.

9.2 8-Fact V1 Baseline Benchmark

The earliest complete benchmark uses an 8-fact LoRA adapter trained on Run-001 (180 tasks) and evaluated on the Run-002 holdout (50 samples). This line predates the 13-fact ontology and is included for historical reference.

Table 9.2: 8-fact v1 benchmark: Qwen3.5-9B Base vs. LoRA-v1 on Run-002 holdout (50 samples, EN).

Metric	Base	LoRA-v1	Δ
JSON valid rate	1.0	1.0	—
Schema valid rate	1.0	1.0	—
Fact accuracy	0.7600	0.9150	+0.1550
Macro F_1	0.4241	0.5847	+0.1605
Seen F_1	0.4714	0.8380	+0.3666
Sample exact match	0.06	0.36	+0.30
Critical false positives	13	15	+2

The LoRA adapter improves fact-level metrics substantially but increases critical false positives from 13 to 15, confirming the 8-fact v1 ontology’s structural weakness: compound targets such as `ins_go` were not visually verifiable from single frames and produced systematic false-positive errors on completion-type facts.

9.3 Dataset Fact Distributions

Table 9.3 reports per-fact `seen` counts for the two main training sources (Run-003, Run-005) and the two 13-fact holdout sets (Run-002 newfacts, Run-004 random). Counts are drawn from the corresponding `stats.json` files.

The Run-005 composition rebalance successfully improved coverage of underrepresented facts: `supt_page_visible` from 20 to 54 combined seen, `fcsmc_page_visible` from 55 to 111, and `ins_grnd_alignment_text_visible` from 58 to 130.

Table 9.3: Per-fact **seen** counts across training and holdout datasets (13-fact v2 ontology).

Fact ID	Run-003 (220)	Run-005 (122)	Run-002 new (50)	Run-
tac_page_visible	46	24	23	
supt_page_visible	20	34	5	
fcs_page_visible	30	40	13	
fcs_page_x_marks_visible	16	18	1	
bit_root_page_visible	18	43	9	
fcsmc_page_visible	55	56	9	
fcsmc_intermediate_result_visible	18	15	1	
fcsmc_in_test_visible	18	19	6	
fcsmc_final_go_result_visible	18	22	2	
hsi_page_visible	106	109	49	
hsi_map_layer_visible	79	37	27	
ins_grnd_alignment_text_visible	58	72	42	
ins_ok_text_visible	12	14	4	

9.4 CLI Command Reference

The **simtutor** package provides the following CLI commands:

- run.** Executes a simulation using a mock environment adapter with a pre-recorded scenario file and writes the event log.
- replay.** Validates an existing event log against a procedure pack, checking step ordering and state-machine consistency.
- score.** Scores an event log using the scoring engine, producing omission counts, state-violation counts, and a total error score.
- record-vlm.** Runs VLM fact extraction on a set of vision observations and records predictions for benchmarking.
- replay-bios.** Replays a raw DCS-BIOS capture through the telemetry pipeline and optionally invokes the help generation loop.
- validate.** Validates JSONL files against a named schema from the schema registry.

All commands accept model configuration overrides (**-model-provider**, **-model-name**, **-model-base-url**, **-model-timeout-s**) and a language switch (**-lang**).

9.5 Model Provider Configuration

Model access is configured through environment variables: `SIMTUTOR_MODEL_PROVIDER` (`openai_compat` | `ollama` | `stub`), `SIMTUTOR_MODEL_BASE_URL`, `SIMTUTOR_MODEL_NAME`, `SIMTUTOR_MODEL_TIMEOUT_S`, `SIMTUTOR_MODEL_API_KEY` (required for `openai_compat`), `SIMTUTOR_MODEL_ENABLE_MULTIMODAL` (toggles base64 image encoding in requests), and `SIMTUTOR_LANG` (`zh` | `en`). The live runtime validates all required configuration at startup and redacts API keys from all log output.

9.6 Replay Evaluation Suite

Table 9.4 lists the five replay evaluation cases in the `fa18c_startup_v04` suite.

Table 9.4: Replay evaluation cases.

Case ID	Expected step	Vision	Expected target
<code>noop_2min</code>	S01	unavailable	<code>battery_switch</code>
<code>batteryon_2min</code>	S02	unavailable	<code>fire_test_switch</code>
<code>Batteryon_enginGenoff_2min</code>	S01	unavailable	<code>generator_left_switch</code>
<code>fcs_reset_fcs_bit_2min</code>	S18	available	<code>fcs_bit_switch</code>
<code>ins_2min</code>	S12	available	<code>ins_mode_knob</code>

Cases with `vision_status = available` include per-frame sidecar files (`frames.jsonl`) in the corresponding DCS Saved Games directory, enabling replay of vision-dependent procedure steps without a live DCS instance.

9.7 Supplementary Architecture and Export Diagrams

The main chapters use simplified diagrams to support the research argument. Figures 9.1, 9.2, and 9.3 retain additional engineering detail for reproducibility and operational context.

9.8 VR Pilot Study — Supplementary Data

Table 9.5 lists participant experience profiles.

Table 9.6 lists the specific steps not completed by each without-tutor participant.

Harness and evidence validation

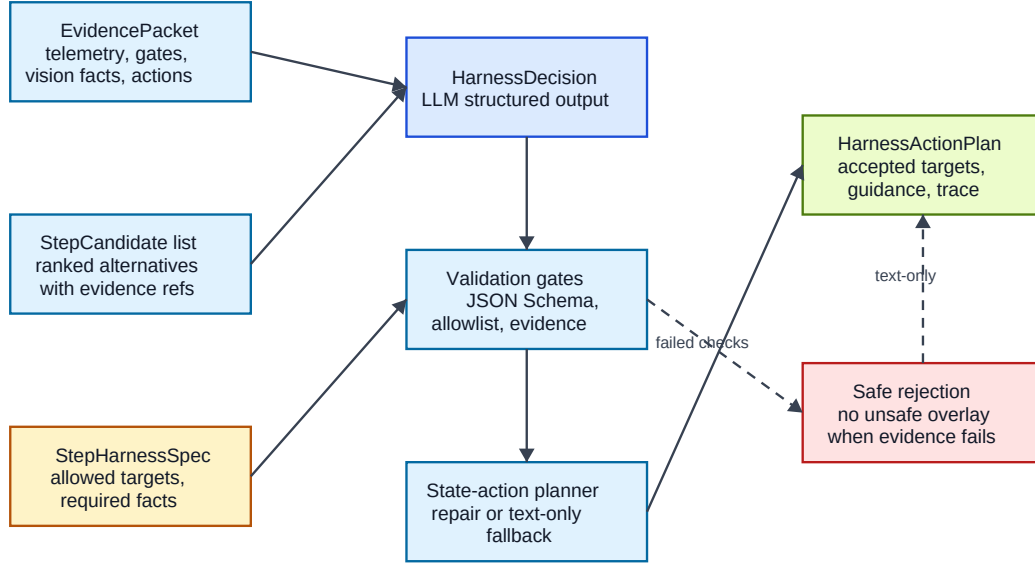


Figure 9.1: Harness and evidence validation. Candidate tutor actions are checked against schema constraints, UI allowlists, evidence consistency, and procedure state before an overlay action plan is accepted; unsafe or under-grounded actions are repaired, rejected, or downgraded to text-only guidance.

Table 9.5: Pilot participant experience profiles (self-reported).

P	Condition	DCS exp.	Flight sim.	VR exp.	Quest 3	Group
P01	without_tutor	frequent	frequent	frequent	frequent	intermediate
P02	without_tutor	intermediate	intermediate	frequent	frequent	—
P03	without_tutor	novice	novice	intermediate	frequent	beginner
P04	with_tutor	intermediate	intermediate	frequent	frequent	intermediate
P05	with_tutor	intermediate	intermediate	frequent	frequent	—
P06	without_tutor	novice	novice	intermediate	frequent	beginner
P07	without_tutor	novice	novice	intermediate	frequent	beginner
P08	with_tutor	novice	novice	frequent	frequent	—
P09	with_tutor	novice	novice	intermediate	intermediate	beginner

— = not recorded.

Detailed experiment export pipeline

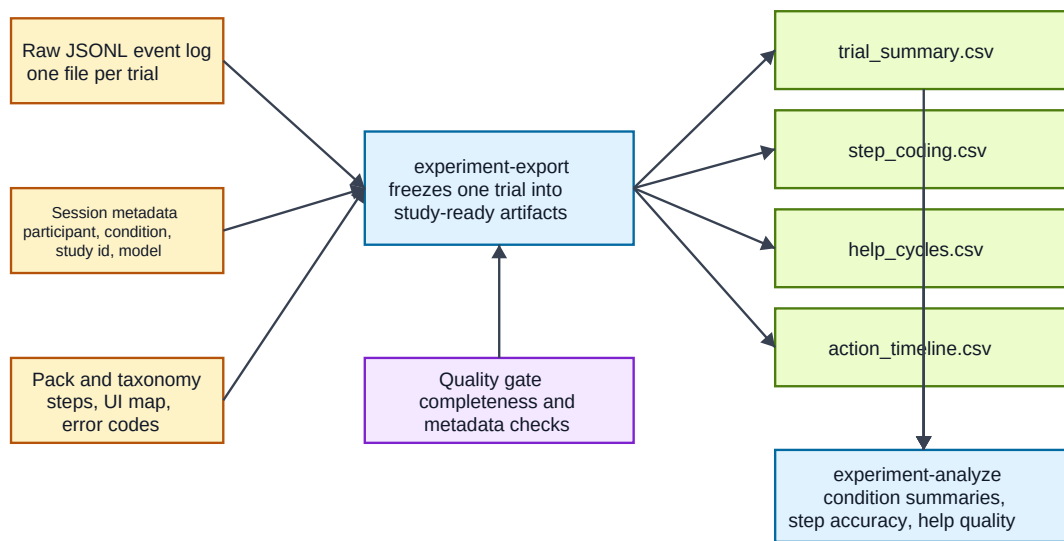


Figure 9.3: Detailed experiment export pipeline. Runtime event logs and trial metadata are transformed into participant-level summaries, step coding, help-cycle records, quality gates, and study-level analysis tables.

Table 9.7 reports the auto-coded error counts (before manual review) alongside the manual error counts, illustrating the magnitude of auto-to-manual correction in this pilot.

Table 9.7: Auto-coded vs. manual error counts per participant.

P	Condition	Auto OM	Auto CO	Manual OM	Manual CO	Total manual
P01	without_tutor	4	3	8	0	10
P02	without_tutor	0	3	0	0	1
P03	without_tutor	0	4	7	0	11
P06	without_tutor	0	3	2	0	4
P07	without_tutor	1	9	11	0	11
P04	with_tutor	0	4	0	3	7
P05	with_tutor	0	3	0	3	6
P08	with_tutor	0	3	0	0	0
P09	with_tutor	0	3	0	3	5

Auto-coded errors are from the automated pipeline (Auto_Error_* columns in `step_coding.csv`). Manual errors are from the human-reviewed columns (Error_*). P08's auto errors were all overridden to zero by manual review (see data-quality note in Section 6.3).